Fusion Leap AI Growth — Plan de développement et prompts quotidiens

Nov 4, 2026 · @Fusion Leap

0. Hypothèses et mode d'emploi

Le MVP complet tient en **63 jours ouvrés (378 h)**, du lundi 9 novembre 2026 au mardi 9 février 2027, en six phases calquées sur le §12 de la spécification. Aucune ligne de code n'est écrite avant la validation du plan au Jour 0 (gate G0).

Hypothèse	Valeur retenue	Si elle change
Équipe	1 développeur·se à temps plein + 1 AI Coding Agent (Claude Code recommandé)	Avec 2 développeurs, compter environ −35 % sur les phases 2 à 4
Capacité	6 h productives/jour, 5 jours/semaine	À 4 h/jour, la fin glisse vers fin mars 2027
Calendrier	Jours fériés marocains exclus : 18 nov, 1er jan, 11 jan, 14 jan	Ajouter un jour par congé personnel
Hypothèse fonctionnelle	Celle du §14 : CRM interne source de vérité, un fournisseur e-mail, numéro WhatsApp de test officiel, Salla en mode marketing (sans lecture de boutique), plans SaaS activés manuellement	Chaque réponse aux questions Q1–Q5 est traitée par un ADR au Jour 1
Stack	Next.js App Router, Supabase (Auth, Postgres RLS, Storage), worker Node séparé, TypeScript strict, pnpm monorepo	Toute déviation = ADR + accord explicite
Langue	Prompts en français ; code, identifiants, commits et noms de tables en anglais	—
Marge	Chaque dernier jour de phase sert de checkpoint et de tampon	Un retard > 1 jour déclenche la procédure du §13

Mode d'emploi quotidien

1. Ouvrir la session de l'agent avec le **prompt de démarrage** (§12).
2. Coller le **prompt du jour** (§5 à §10). L'agent propose un mini-plan et attend votre « OK ».
3. Vérifier les **critères de validation** du jour avant de cocher la journée.
4. Terminer avec le **prompt de clôture** (§12), qui met à jour le journal et le pourcentage réel.
5. En fin de phase, lancer le **prompt d'audit de conformité** (§15) avant de passer la gate.

1. Analyse de la spécification v1.0

La spécification fait de l'isolation des tenants le critère n°1 : tout le reste (CRM, canaux, campagnes, IA) n'a de valeur que si une entreprise A ne peut jamais lire ni modifier les données d'une entreprise B. Le plan s'aligne donc sur l'ordre imposé au §5 : isolation et adhésion → CRM et capture → boîte de réception → campagnes → IA → rapports → intégrations élargies.

1.1 Invariants non négociables

Ces règles sont reprises mot pour mot dans le fichier de règles de l'agent (§2) et vérifiées à chaque checkpoint.

1. **Pas de n8n**, ni dans le produit ni dans l'architecture ; tâches de fond dans les services applicatifs et un worker séparé (spec §1).
2. **MCP n'est pas une file de jobs ni un moteur de permissions** ; c'est une passerelle d'outils optionnelle, lecture seule d'abord (§1, §6).
3. **`organization_id uuid not null` + index** sur toute table tenant ; PK en UUID (§7).
4. **RLS sur toute table exposée** à la Data API, avec politiques séparées lecture/écriture/suppression, `USING` et `WITH CHECK`, contrôle du rôle (§7.2, §11).
5. **Jamais `user_metadata`** pour une décision d'autorisation ; la source du rôle est `organization_members` (§7.2).
6. **Double contrôle** : couche service + base de données ; le serveur ne fait jamais confiance à un `organization_id` envoyé seul (§8, §11).
7. **`service_role` uniquement côté serveur/worker**, jamais dans le navigateur ni dans une variable `NEXT_PUBLIC_*` (§6).

8. **Aucun envoi externe sans approbation d'un utilisateur habilité** dans le MVP ; l'IA ne fait que des brouillons (§5.7, §9.5).
9. **Webhooks** : signature vérifiée, déduplication, ACK rapide, traitement différé ; le tenant est déduit de l'identité du compte fournisseur, jamais d'un champ libre (§8, §11).
10. **Contrôle final avant chaque appel fournisseur** : consentement, `do_not_contact`, template, limites de débit (§9.2, §9.3).
11. **WhatsApp uniquement via Cloud API officielle**, jamais WhatsApp Web (§9.3).
12. **Secrets chiffrés via `secret_ref`**, jamais de jeton brut en base, ni dans les logs, ni dans `audit_logs` (§7, §11).
13. **Toute modification post-approbation** d'une campagne la renvoie en `pending_approval` (§10).
14. **Les chiffres affichés indiquent leur source** (direct, importé, calculé) ; « non disponible » quand le coût est inconnu (§10).
15. **Les tests RLS ne sont jamais reportés** à la fin du projet (§12).

1.2 Périmètre MVP et hors périmètre

Dans le MVP (§5)	Hors MVP (§1.2)
Auth, organisations, invitations, changement d'organisation, rôles, plans et quotas	Création ou publication automatique d'apps/thèmes Salla
Projets, produits, audiences, offres, assets	Achat d'annonces ou ajustement automatique des budgets
CRM interne léger : contacts, leads, deals, activités, tâches, assignation, étapes configurables	Campagnes WhatsApp de masse sans consentement ni template approuvé
Capture REST sécurisée + import CSV avec aperçu et déduplication	Agent qui écrit aux clients ou change une étape de deal sans règle ni approbation
Inbox unifiée e-mail + WhatsApp, état et assignation	Facturation automatique complète, marketplace d'extensions
Campagnes : objectif, segment, canal, contenu, planning, approbation, UTM, résultats de base	

Dans le MVP (§5)	Hors MVP (§1.2)
Agents IA en mode brouillon, résumé, qualification, prochaine étape	
Tableau de bord, journal d'audit, alertes d'échec d'intégration	
Un adaptateur CRM (interne par défaut)	

1.3 Écarts et zones à trancher dans la spécification

La lecture croisée du modèle de données (§7) et des parcours (§4, §5, §9) révèle des entités utilisées mais non définies. Chacune devient un ADR au Jour 1 ; l'agent ne doit pas les inventer sans accord.

#	Constat	Section	Proposition par défaut
E1	« Étapes configurables » mais aucune table de pipeline	§5.3, §7	Tables `pipelines` et `pipeline_stages` par organisation
E2	Tâches citées au MVP, pas de table `tasks`	§5.3	`activities.type = 'task'` avec `due_at` / `completed_at`
E3	`landing_page_id` utilisé, pas de table	§4.1	Champ texte libre dans `leads.utm` en MVP
E4	Reply Agent s'appuie sur une « base de connaissance » non modélisée	§9.5	Table `knowledge_items` par organisation/projet
E5	Templates WhatsApp sans table dédiée	§9.3	Table `message_templates` (provider, name, version, language, status)
E6	Consommation suivie par cycle, pas de compteurs	§10	Table `usage_counters (organization_id, period, metric, value)`
E7	Session de support Platform Admin limitée dans le temps, non modélisée	§2	Table `support_sessions` + audit obligatoire
E8	Règles d'assignation et mapping de champs CRM non modélisés	§9.1	Tables `assignment_rules` et `crm_field_mappings`

#	Constat	Section	Proposition par défaut
E9	`audience_snapshot_id` (campaigns) vs `segment_snapshot_id` (messages)	§7, §10	Un seul nom : `segment_snapshot_id`, table `segment_snapshots`
E10	`plan_id` sur organizations et `plan` sur subscriptions : double source	§7	`subscriptions` source unique ; `plan_id` retiré ou dérivé
E11	`deals`, `messages`, `activities` sans `organization_id` listé	§7	Ajout systématique (règle « toute table tenant »)
E12	Contrainte « lead et projet de la même organisation » non spécifiée techniquement	§7.1	FK composites `(organization_id, project_id)`

1.4 Matrice de traçabilité

Chaque critère d'acceptation du §13 est relié à la phase qui le construit et au test qui le prouve. Elle sert de référence aux checkpoints.

Critère §13	Construit en	Prouvé par	Jour de preuve
1. Isolation des entreprises	Phase 1	Suite RLS + T1 + T5	J15, puis J60
2. Rôles	Phase 1	Matrice RBAC testée rôle × action	J15
3. Capture des leads	Phase 2	Tests d'intégration dédup + T2	J26
4. CRM	Phase 2	Parcours lead → deal gagné	J27
5. E-mail	Phase 3	Envoi test + désabonnement (T3)	J40
6. WhatsApp	Phase 3	T4 : webhook dupliqué, panne temporaire	J40
7. Campagnes	Phase 4	Machine d'états + pause/stop	J53
8. IA	Phase 4	Brouillon non envoyé + test d'injection	J53
9. SaaS	Phase 5	Onboarding sans intervention base	J60

Critère §13	Construit en	Prouvé par	Jour de preuve
10. Exploitation	Phases 3 et 5	Dead letter visible + restauration testée	J58

2. Fichier de règles permanent de l'agent (CLAUDE.md)

Ce fichier, placé à la racine du dépôt, est relu par l'agent à chaque session : il évite de répéter les contraintes dans chaque prompt. Copiez-le tel quel dans `CLAUDE.md` (ou `AGENTS.md` pour un autre agent), et placez la spécification dans `docs/spec/`.

```
# CLAUDE.md — Fusion Leap AI Growth

## Rôle
Tu es l'ingénieur·e logiciel senior qui développe Fusion Leap AI Growth,
une plateforme SaaS multi-tenant. Tu travailles sous la direction de la
responsable technique. Tu ne prends aucune décision d'architecture seul.

## Sources de vérité (par ordre de priorité)
1. docs/spec/Fusion-Leap-AI-Growth-Technical-Spec.md (spec v1.0, 3 nov 2026)
2. docs/adr/*.md (décisions validées qui précisent ou amendent la spec)
3. docs/plan/development-plan.md (plan validé au Jour 0)
4. docs/progress/journal.md (état réel du projet)
Si le code et ces documents divergent, les documents gagnent.
Si la spec est ambiguë ou muette : ARRÊTE-TOI et pose la question.
N'invente jamais une table, un champ, un endpoint ou un fournisseur
absent de la spec ou d'un ADR validé.

## Architecture imposée (spec §6)
- apps/web : Next.js App Router (UI, API routes, webhooks, session)
- apps/worker : worker Node séparé (jobs, sync, envois, agent runs)
- packages/domain : types, validation zod, règles métier pures
- packages/adapters : CRM, email, WhatsApp, Salla, AI (interface
  connect / validate / send / receive / refresh / disconnect)
- packages/permissions : RBAC + policy checks
- supabase/migrations : schéma + RLS
- docs : ADR, API, runbooks, journal

## Interdits absolus
- n8n, sous quelque forme que ce soit.
```

```
- MCP comme file de jobs ou moteur de permissions.
- service_role ou jeton OAuth côté navigateur ou dans NEXT_PUBLIC_*.
- user_metadata pour une décision d'autorisation.
- Table tenant sans organization_id NOT NULL + index + RLS.
- Faire confiance à un organization_id venant du client sans le
  comparer au contexte d'adhésion.
- Envoi externe (email, WhatsApp) sans approbation humaine habilitée
  et sans contrôle final consentement / do_not_contact / template.
- WhatsApp Web non officiel.
- Secrets, jetons ou corps de messages sensibles dans les logs ou
  dans audit_logs.
- Campagne longue exécutée dans une requête HTTP.
- Modifier une migration déjà appliquée : créer une nouvelle migration.

## Protocole de tâche (obligatoire pour chaque prompt)
1. LIRE : relis les sections de spec et ADR citées dans le prompt.
2. PLANIFIER : propose un plan court (fichiers touchés, migrations,
   tests, risques) puis ATTENDS mon « OK ». Ne code rien avant.
3. IMPLÉMENTER : petits commits, un sujet par commit.
4. TESTER : écris les tests avant ou avec le code ; lance
   pnpm lint && pnpm typecheck && pnpm test (+ pnpm test:rls si
   la base change). Tout doit être vert.
5. DOCUMENTER : mets à jour docs/ (API, ADR, runbook) et CHANGELOG.md.
6. RAPPORTER : termine par le rapport standard ci-dessous.

## Rapport standard de fin de tâche
- Fait : liste des livrables, avec chemins de fichiers
- Tests : commandes lancées + résultat (nombre de tests, échecs)
- Écarts à la spec : aucun, ou liste justifiée
- Questions ouvertes / blocages
- Prochaine étape proposée

## Définition de « Terminé »
- Critères de validation du jour atteints et démontrés
- Lint, typecheck, tests unitaires, intégration et RLS verts
- Migration réversible (fichier down ou procédure documentée)
- Pas de TODO caché : tout reste à faire est listé dans le rapport
- Documentation et journal mis à jour

## Conventions
- TypeScript strict, pas de any non justifié
- Validation zod sur toute entrée (API, webhook, job payload, env)
- Erreurs normalisées { code, message, request_id }
```

```
- Idempotency-Key sur toute mutation rejouable
- snake_case en base, camelCase en TS, kebab-case pour les fichiers
- Commits Conventional Commits : feat(scope): ..., fix, test, docs
- Une branche par jour : day-XX-sujet ; PR avec la checklist du jour
- Données de test uniquement ; jamais de données client réelles
  envoyées à un modèle externe

## Commandes
pnpm dev | pnpm lint | pnpm typecheck | pnpm test | pnpm test:rls
pnpm test:e2e | pnpm db:reset | pnpm db:migrate | pnpm worker:dev
```

3. Jour 0 — Analyse et proposition de plan (lun 9 nov, 6 h)

Le Jour 0 ne produit aucun code : l'agent lit tout, restitue sa compréhension, liste les écarts et propose un plan que vous validez ou corrigez. Le développement ne démarre qu'après votre « PLAN VALIDÉ ».

Préparation (30 min, par vous) : créer le dépôt vide, y placer `CLAUDE.md` (§2), `docs/spec/` (le .md et le PDF de la spec) et `docs/plan/` (ce document exporté en Markdown).

Tâche	Durée
Préparation du dépôt	0,5 h
Prompt 0-A : analyse complète par l'agent	1,5 h
Relecture de la restitution, réponses aux questions	1,5 h
Prompt 0-B : plan détaillé	1 h
Corrections, validation, commit des documents	1,5 h

Livrables : `docs/analysis/spec-understanding.md`, `docs/analysis/gaps-and-questions.md`, `docs/plan/development-plan.md`, `docs/progress/journal.md` initialisé.

Critères de validation (gate G0)

- [ ] La restitution reprend les 15 invariants du §1.1 sans en oublier ni en inventer
- [ ] Les 12 écarts E1–E12 sont identifiés (ou d'autres, justifiés par section)
- [ ] Le plan respecte l'ordre de priorité du §5 de la spec
- [ ] Chaque critère d'acceptation §13 est relié à une phase et à un test

- [ ] Aucune technologie hors stack n'est proposée sans ADR
- [ ] Vous avez écrit « PLAN VALIDÉ » dans la conversation

Prompt 0-A — Analyse (à coller tel quel)

```
JOUR 0 — ANALYSE. Tu ne dois écrire AUCUN code aujourd'hui.

Lis intégralement, dans cet ordre :
1. CLAUDE.md
2. docs/spec/Fusion-Leap-AI-Growth-Technical-Spec.md (et le PDF pour
   les schémas : figures 1 à 14)
3. docs/plan/ (plan de développement proposé par la responsable technique)

Produis ensuite docs/analysis/spec-understanding.md contenant :
A. Le produit en 10 lignes : pour qui, quoi, pourquoi SaaS multi-tenant.
B. Les rôles (§2) en matrice rôle × action (lire, créer, modifier,
   envoyer, approuver, gérer intégrations, gérer membres), avec la
   section source de chaque case.
C. Le modèle de données (§7) : chaque table, ses champs, ses relations,
   la colonne organization_id, les contraintes d'unicité et d'intégrité.
D. Les parcours : capture lead (§9.1), email (§9.2), WhatsApp (§9.3),
   Salla (§9.4), agents IA (§9.5), cycle de vie campagne (§10).
E. Les invariants de sécurité et de conformité (§6, §7.2, §8, §11) sous
   forme de liste numérotée vérifiable.
F. Les 10 critères d'acceptation (§13) et les 5 scénarios réalistes.

Produis docs/analysis/gaps-and-questions.md contenant :
- Chaque entité ou règle utilisée dans la spec mais non modélisée
  (référence de section obligatoire), avec une proposition par défaut.
- Chaque incohérence de nommage ou de source de vérité.
- Les questions Q1–Q5 du §14 et leur impact précis sur le plan.
- Les risques techniques classés (probabilité × impact).

Règles : cite la section pour chaque affirmation ; distingue clairement
ce que dit la spec et ce que tu proposes ; n'invente aucune exigence.
Termine par le rapport standard et attends mes réponses.
```

Prompt 0-B — Plan détaillé (après vos réponses)

```
JOUR 0 — PLAN. Toujours AUCUN code.

En tenant compte de mes réponses et du plan de développement fourni
dans docs/plan/, produis docs/plan/development-plan.md :
1. Les 6 phases (0 à 5) avec dates, gates et conditions de passage
   exactes du §12 de la spec.
2. Pour chaque jour J1 à J62 : objectif, tâches estimées en heures
   (total 6 h), fichiers/migrations attendus, tests, critère de
   validation. Signale chaque jour dont l'estimation te paraît
   irréaliste et propose un découpage.
3. Le graphe de dépendances entre tâches (ce qui bloque quoi).
4. La liste des ADR à rédiger en Phase 0, avec la décision recommandée.
5. La stratégie de test (unit, intégration, RLS, E2E) et les outils.
6. Les dépendances externes (Meta, Google, Salla, fournisseur email)
   et le plan B si elles ne sont pas prêtes à temps.
7. Une matrice de traçabilité : exigence spec → jour → test.

Compare ton plan au plan fourni : liste les différences et justifie-les.
N'élargis pas le périmètre MVP. Termine par le rapport standard et
attends le message « PLAN VALIDÉ » avant toute implémentation.
```

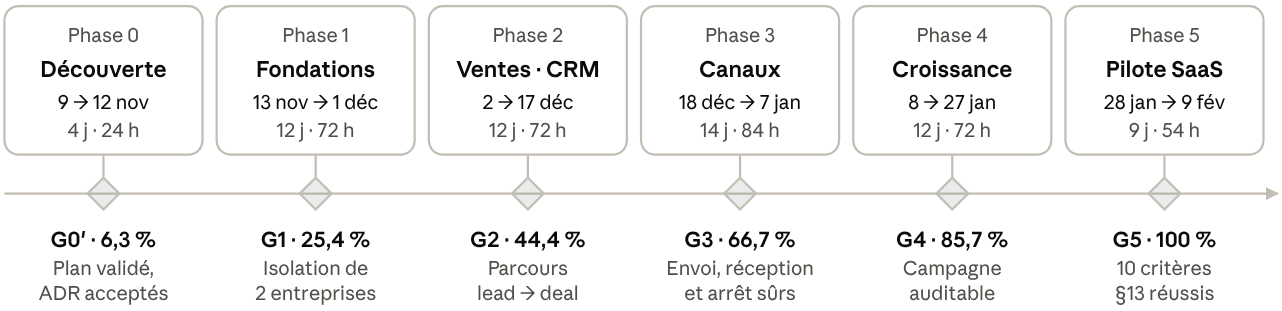
4. Vue d'ensemble du planning

Six phases, six gates : chaque phase ne démarre que si la condition de passage de la précédente est démontrée, comme l'exige le §12 de la spec. Le pourcentage cumulé est la cible ; l'avancement réel se calcule chaque soir (§12 de ce plan).

Phase	Dates	Jours	Heures	% cumulé cible	Gate (condition de passage, spec §12)
0 — Découverte technique	9 → 12 nov	J0–J3 (4)	24 h	6,3 %	G0/G0′ : plan validé, ADR approuvés, sources de vérité arrêtées
1 — Fondations	13 nov → 1er déc	J4–J15 (12)	72 h	25,4 %	G1 : test d'isolation de deux entreprises réussi
2 — Ventes / CRM	2 → 17 déc	J16–J27 (12)	72 h	44,4 %	G2 : parcours lead → deal fonctionnel
3 — Canaux	18 déc → 7 jan	J28–J41 (14)	84 h	66,7 %	G3 : envoi, réception et arrêt sûrs

Phase	Dates	Jours	Heures	% cumulé cible	Gate (condition de passage, spec §12)
4 — Croissance	8 → 27 jan	J42–J53 (12)	72 h	85,7 %	G4 : campagne complète et auditable
5 — Pilote SaaS	28 jan → 9 fév	J54–J62 (9)	54 h	100 %	G5 : les 10 critères d'acceptation §13 réussis
Total		**63 jours**	**378 h**		

Six phases, six gates : aucune phase ne démarre sans preuve



feuille de route · 6 phases, 6 gates

Chaque losange est une gate du §12 de la spec ; le pourcentage indique l'avancement cumulé cible à cette date.

Lecture des fiches journalières (§5 à §10). Chaque jour indique : la date, la durée, le pourcentage cumulé cible à atteindre le soir, l'objectif, les tâches chiffrées, le livrable, les critères de validation (cases à cocher) et le prompt exact. Tous les prompts supposent que `CLAUDE.md` est en place : l'agent applique donc le protocole « plan → OK → code → tests → doc → rapport » sans qu'il faille le répéter.

5. Phase 0 — Découverte technique (J1–J3)

La Phase 0 fige les décisions qui conditionnent tout le reste : modèle de données réconcilié, fournisseurs retenus, menaces identifiées. Gate G0′ : ADR approuvés par vous et sources de vérité arrêtées.

J1 · mar 10 nov · 6 h · cible 3,2 %

Objectif : trancher les écarts E1–E12 et figer le modèle de données logique.

Tâches : ADR 001–006 (stack, monorepo, file de jobs, stockage des secrets, multi-tenancy, conventions) 2,5 h · ADR 007 modèle de données réconcilié 2 h · diagramme ERD Mermaid 1 h · revue 0,5 h.

Livrable : `docs/adr/001…007-*.md`, `docs/architecture/erd.md`.

- [ ] Chaque ADR suit le format Contexte / Décision / Alternatives / Conséquences / Section de spec
- [ ] Chaque table tenant a `organization_id`, ses index et sa stratégie RLS décrite
- [ ] E1–E12 sont tous tranchés

```
JOUR 1 — ADR ET MODÈLE DE DONNÉES. Pas de code applicatif.
Sources : spec §6, §7, §7.1, §7.2, §11 ; docs/analysis/gaps-and-questions.md.

1. Rédige docs/adr/ au format MADR : 001-stack, 002-monorepo-pnpm-turbo,
   003-job-queue (table jobs Postgres + worker avec lease, FOR UPDATE
   SKIP LOCKED, vs file managée : recommande et justifie), 004-secrets
   (Supabase Vault ou chiffrement applicatif, secret_ref), 005-multi-
   tenancy (organization_id partout, FK composites, fonctions d'aide RLS
   dans un schéma private non exposé), 006-conventions.
2. ADR 007-data-model : réconcilie la spec avec les écarts E1–E12 de
   gaps-and-questions.md. Pour chaque table : colonnes, types, NOT NULL,
   index, contraintes uniques, FK composites (organization_id, id),
   politiques RLS prévues par rôle.
3. docs/architecture/erd.md : diagramme Mermaid erDiagram complet.
N'ajoute aucune table hors spec sans la marquer « ajout ADR » avec sa
section d'origine. Rapport standard puis attends ma validation.
```

J2 · mer 11 nov · 6 h · cible 4,8 %

Objectif : documenter chaque intégration et le flux de données (questions Q1–Q5 du §14).

Tâches : fiche CRM existant 1 h · fiche fournisseur e-mail 1,5 h · fiche WhatsApp Cloud API 1,5 h · fiche Salla 1 h · diagramme de flux de données 1 h.

Livrable : `docs/integrations/{crm,email,whatsapp,salla}.md`, `docs/architecture/data-flow.md`, ADR 008-email-provider.

- [ ] Chaque fiche cite la documentation officielle consultée et sa date
- [ ] Les limites de débit, signatures webhook et scopes OAuth sont notés
- [ ] Le plan B de chaque intégration est écrit

```
JOUR 2 — DÉCOUVERTE DES INTÉGRATIONS. Pas de code applicatif.
Sources : spec §9.1–§9.4, §14, §15 (liens officiels).

Pour chaque intégration (CRM Fusion Leap, fournisseur email, WhatsApp
Business Platform Cloud API, Salla), crée docs/integrations/<nom>.md :
- Mode d'authentification, scopes minimaux, rotation/refresh des jetons
- Endpoints utilisés (envoi, réception, statuts) et format des webhooks
- Vérification de signature, déduplication, limites de débit
- Comptes de test / sandbox nécessaires et qui doit les créer
- Exigences de conformité (consentement, templates, désabonnement)
- Plan B si l'accès n'est pas prêt à temps
Vérifie chaque point dans la documentation officielle (spec §15) et
note l'URL + la date ; marque « À CONFIRMER » ce que tu ne peux pas
vérifier. Puis docs/architecture/data-flow.md (Mermaid flowchart) et
ADR 008-email-provider avec 2–3 options comparées. Rapport standard.
```

J3 · jeu 12 nov · 6 h · cible 6,3 %

Objectif : modèle de menaces, stratégie de test, backlog final ; gate G0′.

Tâches : threat model STRIDE (multi-tenant, webhooks, IA) 2 h · stratégie de test 1,5 h · backlog J4–J62 en issues 1,5 h · checkpoint 1 h.

Livrable : `docs/security/threat-model.md`, `docs/testing/strategy.md`, backlog d′issues, entrée de journal.

- [ ] Chaque menace est reliée à un contrôle et à un test
- [ ] Le backlog couvre chaque jour du plan et chaque critère §13
- [ ] Vous avez approuvé ADR 001–008 (statut « Accepted »)

```
JOUR 3 — MENACES, TESTS, BACKLOG. Pas de code applicatif.
Sources : spec §8, §9.5, §11, §13 ; docs/adr/ ; docs/plan/.

1. docs/security/threat-model.md : analyse STRIDE ciblée sur fuite
   inter-tenant, élévation de rôle, webhooks forgés ou rejoués, jetons
   OAuth, injection de prompt via messages entrants, envoi non consenti.
   Pour chaque menace : contrôle prévu + test qui le prouve.
2. docs/testing/strategy.md : pyramide de tests, outils (Vitest, pgTAP
   ou tests SQL via supabase test db, Playwright, MSW pour les
   fournisseurs), données de test, seuils de couverture, exécution CI.
3. Génère le backlog J4–J62 sous forme de fichiers docs/backlog/JXX.md
   (ou issues GitHub si gh est configuré) à partir du plan validé.
4. Lance l'audit de conformité Phase 0 (prompt §15 du plan).
Rapport standard. Liste les ADR encore « Proposed ».
```

6. Phase 1 — Fondations (J4–J15)

La Phase 1 livre le socle multi-tenant : dépôt, environnements, Auth, organisations, adhésions, RLS, couche service, audit, jobs et quotas. Gate G1 : le test d'isolation entre deux entreprises passe sur l'UI, l'API, la RLS et le worker.

J4 · ven 13 nov · 6 h · cible 7,9 %

Objectif : monorepo opérationnel avec CI. **Tâches** : pnpm + Turborepo et squelette des 7 dossiers du §6 (2 h) · TS strict, ESLint, Prettier, Vitest (1,5 h) · CI GitHub Actions lint/typecheck/test (1,5 h) · README (1 h). **Livrable** : dépôt buildable, CI verte.

- [ ] `pnpm install && pnpm lint && pnpm typecheck && pnpm test` passent en local et en CI
- [ ] Arborescence identique au §6 de la spec ; versions épinglées et lockfile commité

```
JOUR 4 — SCAFFOLDING DU MONOREPO. Sources : spec §6 (structure du dépôt,
versions épinglées), ADR 001, 002, 006.
Crée le monorepo pnpm + Turborepo : apps/web (Next.js App Router, TS
strict), apps/worker (Node TS), packages/domain, packages/adapters,
packages/permissions, supabase/, docs/. Configure TS strict partagé,
ESLint, Prettier, Vitest, scripts racine (dev, lint, typecheck, test,
test:rls, test:e2e, db:*, worker:dev). Ajoute .github/workflows/ci.yml
(lint, typecheck, test). Épingle les versions exactes et vérifie les
versions actuelles stables dans la doc officielle. Aucune feature métier.
```

J5 · lun 16 nov · 6 h · cible 9,5 %

Objectif : Supabase local et environnements séparés. **Tâches** : Supabase CLI + projet local (1,5 h) · validation zod des variables d'environnement, séparation serveur/client (2 h) · fichiers `.env.example` dev/staging/prod (1 h) · garde-fou CI anti-fuite `NEXT_PUBLIC_*` (1,5 h). **Livrable** : `pnpm db:reset` fonctionne, module `env` typé.

- [ ] L'app refuse de démarrer si une variable manque
- [ ] Un test CI échoue si `service_role` apparaît dans le bundle client ou une variable publique

```
JOUR 5 — ENVIRONNEMENTS ET SECRETS. Sources : spec §6 (environnements
dev/staging/production, secrets), §11 ; ADR 004.
1. Initialise Supabase CLI en local (supabase/config.toml), scripts
   db:reset, db:migrate.
2. packages/domain/env : schémas zod séparés serverEnv / clientEnv ;
   le service role n'est lisible que côté serveur et worker.
3. .env.example par environnement, sans valeur réelle.
4. Script CI qui échoue si une clé sensible est préfixée NEXT_PUBLIC_
   ou présente dans le build client (.next/static).
Teste le garde-fou avec une fuite volontaire, puis retire-la.
```

J6 · mar 17 nov · 6 h · cible 11,1 %

Objectif : migrations du noyau tenant. **Tâches** : `organizations`, `organization_members` (enum de rôles §2), `subscriptions`, `usage_counters` (3 h) · schéma `private` + fonctions `is_member(org)`, `has_role(org, roles[])` en `security definer` (2 h) · seed de dev (1 h). **Livrable** : migrations + seed.

- [ ] Les 6 rôles du §2 existent comme enum ; clé unique `(organization_id, user_id)`
- [ ] Les fonctions d'aide ne sont pas exposées par la Data API

```
JOUR 6 — MIGRATIONS DU NOYAU. Sources : spec §2, §7, §7.2 ; ADR 005, 007.
Crée les migrations : organizations, organization_members (role enum :
platform_admin, company_owner, company_admin, marketing_manager,
sales_agent, analyst ; status ; invited_by ; unique(organization_id,
user_id)), subscriptions, usage_counters. UUID, created_at, updated_at,
trigger updated_at. Crée le schéma private (non exposé) avec
is_member(org uuid) et has_role(org uuid, roles role[]) en security
definer, search_path fixé, sans boucle RLS sur organization_members.
Seed : 2 entreprises, 1 utilisateur commun avec deux rôles différents.
Pas encore de politiques RLS (demain).
```

J7 · jeu 19 nov · 6 h · cible 12,7 %

Objectif : RLS du noyau et harnais de tests RLS. **Tâches** : politiques select/insert/update/delete par rôle (2,5 h) · révision GRANT et Data API (1 h) · harnais `pnpm test:rls` avec JWT simulés par utilisateur (2,5 h). **Livrable** : suite RLS verte.

- [ ] Utilisateur A ne lit, n'insère, ne modifie ni ne supprime aucune ligne de B (4 opérations testées)
- [ ] Un Analyst ne peut rien écrire ; un non-membre ne voit rien

```
JOUR 7 — RLS DU NOYAU + HARNAIS. Sources : spec §7.2 (exemple RLS et
exigences), §11, §13 critère 1 et 2.
1. Active RLS sur toutes les tables du J6. Politiques séparées par
   opération, USING + WITH CHECK, rôle via private.has_role. Jamais
   user_metadata. Révoque les GRANT inutiles à anon/authenticated.
2. Harnais de test RLS (pgTAP via supabase test db, ou Vitest qui
   se connecte avec des JWT signés par utilisateur de seed).
3. Tests : matrice (utilisateur × entreprise × opération CRUD) +
   tentative de changer organization_id d'une ligne vers l'autre
   entreprise (doit échouer).
Ajoute test:rls à la CI. Chaque nouvelle table devra suivre ce modèle.
```

J8 · ven 20 nov · 6 h · cible 14,3 %

Objectif : authentification SSR. **Tâches** : client Supabase SSR serveur/navigateur selon la doc officielle actuelle (2 h) · middleware de session et routes protégées (1,5 h) · pages inscription, connexion, mot de passe oublié en AR/EN (2,5 h). **Livrable** : parcours d'auth complet.

- [ ] Une route protégée redirige sans session ; la session se rafraîchit côté serveur

- [ ] Interface bilingue arabe (RTL) / anglais

```
JOUR 8 — AUTH SSR. Sources : spec §6 (Next.js, méthode SSR de la doc
Supabase actuelle), §15 (liens Supabase Auth SSR et choix du package).
Relis la doc officielle Supabase SSR (lien §15) avant de coder et cite
la version du package retenu. Implémente : clients serveur/navigateur,
middleware de rafraîchissement de session, groupes de routes (public)
et (app) protégés, pages signup/login/reset. i18n arabe (dir=rtl) et
anglais dès maintenant. Tests : redirection sans session, accès avec
session, déconnexion.
```

J9 · lun 23 nov · 6 h · cible 15,9 %

Objectif : création d'organisation et contexte actif. **Tâches** : `POST /api/orgs` (organisation + adhésion owner dans une transaction) (2 h) · résolution serveur de `active_organization_id` vérifiée contre l'adhésion (2 h) · sélecteur d'organisation (1,5 h) · tests (0,5 h). **Livrable** : flux création/changement d'organisation.

- [ ] Un `organization_id` forgé dans une requête est refusé (403)
- [ ] Changer d'organisation invalide l'accès aux pages de l'ancienne (scénario T5 partiel)

```
JOUR 9 — ORGANISATIONS ET CONTEXTE ACTIF. Sources : spec §2
(active_organization_id), §8 (POST /api/orgs, ne pas faire confiance à
organization_id seul), §13 scénario 5.
Implémente POST /api/orgs (transaction : organization + membre
company_owner + subscription trial), le stockage de l'organisation
active (cookie httpOnly) et un helper serveur getTenantContext() qui
revérifie l'adhésion active à CHAQUE requête. Sélecteur d'organisation
dans l'en-tête. Tests : org forgée → 403 ; deep link vers l'ancienne
organisation après changement → refus.
```

J10 · mar 24 nov · 6 h · cible 17,5 %

Objectif : invitations et RBAC applicatif. **Tâches** : `packages/permissions` matrice rôle × action issue du J0 (2 h) · invitations par e-mail avec jeton expirant (2 h) · page membres : rôle, retrait, transfert de propriété réservé à l'owner (2 h). **Livrable** : gestion des membres.

- [ ] Matrice de permissions testée exhaustivement (rôle × action)
- [ ] Un Company Admin ne peut pas transférer la propriété

```
JOUR 10 — INVITATIONS ET RBAC. Sources : spec §2 (tableau des rôles),
§13 critère 2 ; docs/analysis/spec-understanding.md (matrice B).
1. packages/permissions : can(role, action, resource) pure, générée à
   partir de la matrice du Jour 0 ; tests table-driven exhaustifs.
2. Invitations : table invitations (organization_id, email, role, token
   haché, expires_at), RLS, acceptation qui crée l'adhésion.
3. Page Membres : changer rôle, retirer, transfert de propriété
   (owner uniquement). Chaque action passe par can() ET par la RLS.
```

J11 · mer 25 nov · 6 h · cible 19,0 %

Objectif : couche service standard. **Tâches** : wrapper `withService()` : contexte tenant, zod, permission, `request_id` (2 h) · erreurs normalisées (1 h) · `Idempotency-Key` stockée (1,5 h) · limitation de débit par IP/organisation (1,5 h). **Livrable** : toutes les API passent par ce wrapper.

- [ ] Une mutation rejouée avec la même clé renvoie la même réponse sans double effet
- [ ] Toute erreur porte `code`, `message`, `request_id`

```
JOUR 11 — COUCHE SERVICE. Sources : spec §8 (zod, erreurs unifiées,
limites de débit, request_id, Idempotency-Key), §11.
Crée dans apps/web/lib/service un wrapper withService({ schema,
permission, idempotent }) qui : génère request_id, résout
getTenantContext(), valide l'entrée, vérifie can(), applique la limite
de débit, gère Idempotency-Key (table idempotency_keys avec
organization_id, clé, hash de requête, réponse, expiration), et
normalise les erreurs. Migre POST /api/orgs dessus. Tests unitaires +
intégration (rejeu, clé réutilisée avec un autre corps → 422).
```

J12 · jeu 26 nov · 6 h · cible 20,6 %

Objectif : journal d'audit et logs structurés. **Tâches** : `audit_logs` append-only (pas d'UPDATE/DELETE, même via service) (2 h) · helper `audit()` avec caviardage des secrets (1,5 h) · logger JSON avec `request_id` et filtrage (1,5 h) · branchement sur membres/invitations (1 h). **Livrable** : audit opérationnel.

- [ ] Une tentative d'UPDATE sur `audit_logs` échoue
- [ ] Un test vérifie qu'aucun jeton ou corps de message n'apparaît dans l'audit ni les logs

```
JOUR 12 — AUDIT ET LOGS. Sources : spec §7 (audit_logs), §7.1 (pas de
textes IA ni webhooks sensibles dans audit_logs), §11 (journal non
modifiable, pas de jetons dans les logs).
Migration audit_logs (actor_id, organization_id, action, entity_type,
entity_id, before, after, ip, created_at) : RLS lecture owner/admin,
aucun UPDATE/DELETE (trigger qui lève une exception). Helper audit()
avec liste de champs caviardés. Logger JSON (pino) avec request_id et
redaction. Branche l'audit sur adhésions et invitations. Tests de
caviardage et d'immutabilité.
```

J13 · ven 27 nov · 6 h · cible 22,2 %

Objectif : file de jobs et worker. **Tâches** : table `jobs` (2 h) · worker : claim avec `FOR UPDATE SKIP LOCKED`, lease, heartbeat (2 h) · retry exponentiel + `dead_letter` (1,5 h) · job « noop » de démonstration (0,5 h). **Livrable** : worker qui traite des jobs de façon sûre.

- [ ] Deux workers en parallèle ne traitent jamais le même job
- [ ] Un job qui échoue N fois passe en `dead_letter` ; une `idempotency_key` dupliquée est refusée

```
JOUR 13 — JOBS ET WORKER. Sources : spec §6 (worker : lease, retry,
dead_letter, pas de campagne dans HTTP), §7 (jobs), ADR 003.
Migration jobs (organization_id, kind, payload_ref/payload, status,
attempts, max_attempts, next_run_at, lease_until, idempotency_key
unique par organisation, last_error). apps/worker : boucle de claim
SKIP LOCKED, lease + renouvellement, backoff exponentiel avec jitter,
dead_letter, registre de handlers typés par kind (zod), arrêt propre
(SIGTERM). Le worker utilise service_role MAIS chaque handler reçoit
et applique explicitement organization_id. Tests de concurrence.
```

J14 · lun 30 nov · 6 h · cible 23,8 %

Objectif : plans, quotas, activation manuelle. **Tâches** : définitions des plans et limites §10 (1,5 h) · `assertWithinLimit()` + incrément de `usage_counters` (2 h) · alerte avant seuil et blocage explicite (1 h) · écran Platform Admin d'activation avec audit (1,5 h). **Livrable** : quotas appliqués.

- [ ] Créer un projet au-delà de la limite renvoie une erreur claire
- [ ] Les compteurs de deux entreprises ne se mélangent jamais

```
JOUR 14 — PLANS ET QUOTAS. Sources : spec §10 (statuts trial, active,
past_due, suspended, cancelled ; limites utilisateurs, projets,
contacts, messages/mois, runs IA, intégrations), §2 Platform Admin.
Définitions de plans en configuration versionnée. assertWithinLimit(org,
metric) dans le wrapper service, incrément atomique de usage_counters
par période. Bannière d'avertissement à 80 %, blocage à 100 %. Écran
Platform Admin : changer plan/statut d'une organisation, motif
obligatoire, audit. Tests : dépassement, isolation des compteurs.
```

J15 · mar 1 déc · 6 h · cible 25,4 % · Checkpoint C1 / Gate G1

Objectif : prouver l'isolation. **Tâches** : scénario T1 automatisé (2 h) · scénario T5 (1 h) · corrections (2 h) · audit de conformité Phase 1 (1 h). **Livrable** : rapport `docs/checkpoints/C1.md`.

- [ ] Critère §13-1 démontré sur UI, API, RLS et worker
- [ ] Critère §13-2 démontré pour les rôles disponibles à ce stade
- [ ] Audit de conformité sans écart bloquant

```
JOUR 15 — CHECKPOINT C1 / GATE G1. Sources : spec §12 (condition :
test d'isolation de deux entreprises réussi), §13 critères 1–2,
scénarios 1 et 5.
1. Écris les tests d'intégration T1 (2 entreprises, 1 utilisateur
   commun avec 2 rôles, 2 projets fictifs par entreprise) et T5
   (changement d'organisation + deep link). Couvre UI (Playwright),
   API, RLS et un handler worker.
2. Corrige tout échec sans affaiblir une politique.
3. Exécute le prompt d'audit de conformité (§15 du plan) sur la Phase 1.
4. Rédige docs/checkpoints/C1.md : résultats, écarts, dette, GO/NO-GO.
```

7. Phase 2 — Ventes et CRM (J16–J27)

La Phase 2 construit le CRM interne, source de vérité du MVP : projets, contacts, leads, deals, activités, capture publique, import CSV, assignation et adaptateur CRM. Gate G2 : un lead capturé va jusqu'au deal gagné avec un historique d'activités visible.

J16 · mer 2 déc · 6 h · cible 27,0 %

Objectif : projets et produits. **Tâches** : migrations `projects`, `products` + RLS + tests (2,5 h) · services CRUD avec quota projets (2 h) · schémas zod de `settings` par type de projet (1,5 h). **Livrable** : API projets/produits.

- [ ] Les 4 types (`website_sales`, `salla_app`, `salla_theme`, custom) ont chacun un schéma de settings validé
- [ ] Un produit ne peut pas pointer vers un projet d'une autre entreprise (FK composite)

```
JOUR 16 — PROJETS ET PRODUITS. Sources : spec §4.1–§4.3, §7 (projects,
products), §8 (POST /api/projects, vérification du plan) ; ADR 007.
Migrations projects et products avec organization_id, FK composites
(organization_id, project_id), RLS selon la matrice RBAC, tests RLS.
Schémas zod discriminés par type pour settings : website_sales
(offres, packages, marché cible, questions de qualification, landing
pages), salla_app (nom, problème, segments marchands, lien store,
essai, prix de test, FAQ), salla_theme (lien d'aperçu, captures,
secteurs, atouts, scénarios pub), custom. Services via withService +
assertWithinLimit('projects').
```

J17 · jeu 3 déc · 6 h · cible 28,6 %

Objectif : interface projets. **Tâches** : liste et création (2 h) · formulaires dynamiques par type (2,5 h) · assets via Storage avec politiques tenant (1,5 h). **Livrable** : UI projets bilingue.

- [ ] Les politiques Storage empêchent l'accès aux fichiers d'une autre entreprise (testé)
- [ ] Un Analyst voit les projets sans pouvoir les modifier

```
JOUR 17 — UI PROJETS + STORAGE. Sources : spec §5.2, §11 (Storage
policies, URLs signées), §13 critère 1.
Pages Projets (liste, création, détail) avec formulaires générés à
partir des schémas zod du J16, AR/EN. Bucket Storage privé, chemins
préfixés par organization_id/project_id, politiques Storage basées
sur private.is_member, URLs signées à durée courte. Tests : accès
inter-entreprise refusé, Analyst en lecture seule.
```

J18 · ven 4 déc · 6 h · cible 30,2 %

Objectif : contacts, normalisation, consentement. **Tâches** : `contacts` + `consent_events` + RLS (2 h) · normalisation e-mail et téléphone E.164 (libphonenumber) (1,5 h) · règles de déduplication dans la même entreprise (1,5 h) · tests (1 h). **Livrable** : domaine contacts.

- [ ] Le même e-mail dans deux entreprises crée deux contacts distincts
- [ ] Les formats de téléphone marocains et saoudiens variés se normalisent en E.164

```
JOUR 18 — CONTACTS ET CONSENTEMENT. Sources : spec §7 (contacts,
consent_events), §7 (unique(organization_id, external_source,
external_id), ne pas utiliser l'email comme seul identifiant), §13 critère 3.
Migrations contacts (email_normalized, phone_e164, language, country,
consent_status, do_not_contact) et consent_events (channel, purpose,
source, proof, captured_at, revoked_at) ; index uniques partiels par
organisation. packages/domain : normalizeEmail, normalizePhone (défaut
MA/SA configurable), findDuplicate(org, contact). Tests table-driven
avec formats réels variés.
```

J19 · lun 7 déc · 6 h · cible 31,7 %

Objectif : leads, pipelines, deals. **Tâches** : `pipelines` / `pipeline_stages` (écart E1) (1,5 h) · `leads` (source, stage, score, assignee, utm) (1,5 h) · `deals` (2 h) · tests RLS (1 h). **Livrable** : modèle de vente.

- [ ] Un lead ne peut référencer qu'un projet de sa propre entreprise
- [ ] Les étapes sont configurables par organisation

```
JOUR 19 — LEADS, PIPELINES, DEALS. Sources : spec §5.3, §7 (leads,
deals), §7.1 ; ADR 007 (E1, E11, E12).
Migrations pipelines, pipeline_stages (ordre, type open/won/lost),
leads (contact_id, project_id, source, stage_id, score, assignee_id,
campaign_id nullable, utm jsonb), deals (lead_id, pipeline_stage, value
numeric, currency, expected_close_at, won_at, lost_at), tous avec
organization_id et FK composites. Pipeline par défaut créé avec
l'organisation. Services de transition d'étape avec règles (won → won_at).
Tests RLS + transitions.
```

J20 · mar 8 déc · 6 h · cible 33,3 %

Objectif : activités, tâches, chronologie. **Tâches** : `activities` (types note, appel, e-mail, tâche) (1,5 h) · journalisation automatique des changements d'étape (1,5 h) · chronologie du lead dans l'UI (2 h) · tâches dues (1 h). **Livrable** : fiche lead avec historique.

- [ ] Chaque étape affiche statut, responsable et prochaine action (spec §3)
- [ ] Un Sales Agent ne voit que ses leads assignés

```
JOUR 20 — ACTIVITÉS ET CHRONOLOGIE. Sources : spec §3 (statut,
responsable et prochaine étape visibles à chaque étape), §5.3, §7
(activities), §2 (Sales Agent : ses leads assignés).
Migration activities (lead_id/contact_id, type, owner_id, due_at,
completed_at, metadata) ; création automatique d'une activité à chaque
changement d'étape ou d'assignation. Fiche lead : en-tête (statut,
responsable, prochaine action), chronologie, création de tâche. RLS :
sales_agent limité aux leads où assignee_id = auth.uid().
```

J21 · mer 9 déc · 6 h · cible 34,9 %

Objectif : capture publique sécurisée. **Tâches** : clés de formulaire par projet (1 h) · `POST /api/leads` : origine, rate limit, honeypot, zod (2 h) · transaction contact + lead + consent_event + job d'assignation (2 h) · tests (1 h). **Livrable** : endpoint de capture + snippet HTML.

- [ ] Deux soumissions identiques ne créent qu'un contact (critère §13-3)
- [ ] Source, UTM, `landing_page_id` et texte/version du consentement sont stockés

```
JOUR 21 — CAPTURE DE LEADS. Sources : spec §8 (POST /api/leads), §9.1
étapes 1–3, §4.1 (UTM, landing_page_id, consentement), §13 critère 3.
Table form_keys (organization_id, project_id, clé hachée, origines
autorisées). POST /api/leads public : identifie l'organisation par la
clé (jamais par un champ du corps), vérifie l'origine, rate limit,
honeypot, zod ; dans UNE transaction : upsert contact dédupliqué,
création lead, consent_event avec texte et version ; puis enqueue
d'un job assign_lead. Retourne le résultat de déduplication. Fournis
un snippet HTML d'exemple. Tests d'intégration complets.
```

J22 · jeu 10 déc · 6 h · cible 36,5 %

Objectif : assignation. **Tâches** : `assignment_rules` (E8) round-robin et par projet (2 h) · handler worker `assign_lead` (1,5 h) · file « non assignés » + alerte manager (1,5 h) · tests (1 h). **Livrable** : assignation automatique.

- [ ] Un échec d'assignation place le lead dans la file non assignée et notifie le manager
- [ ] Rejouer le job ne réassigne pas deux fois

```
JOUR 22 — ASSIGNATION. Sources : spec §9.1 étape 4, §5.3 ; ADR 007 (E8).
Migration assignment_rules (organization_id, project_id nullable,
stratégie round_robin|fixed, membres éligibles). Handler worker
assign_lead idempotent ; en cas d'échec : stage « unassigned »,
notification in-app aux company_admin. Vue « Leads non assignés ».
Tests : rejouer le job, membre retiré, aucun membre éligible.
```

J23 · ven 11 déc · 6 h · cible 38,1 %

Objectif : import CSV. **Tâches** : upload et parsing en streaming (1,5 h) · mapping des colonnes + aperçu des 20 premières lignes (2 h) · rapport de déduplication avant validation (1 h) · commit via job worker (1,5 h). **Livrable** : assistant d'import.

- [ ] Rien n'est écrit avant la confirmation de l'aperçu
- [ ] Un fichier de 100 lignes avec doublons et téléphones hétérogènes s'importe correctement

```
JOUR 23 — IMPORT CSV. Sources : spec §5.4 (aperçu et déduplication),
§13 scénario 2.
Assistant en 3 étapes : upload (taille et type vérifiés, Storage
privé), mapping des colonnes vers contact/lead, aperçu + rapport
(nouveaux, doublons, erreurs par ligne). La validation crée un job
import_csv qui réutilise exactement la logique de dédup du J18–J21.
Rapport final téléchargeable. Fixture tests/fixtures/leads-100.csv
avec doublons et formats variés.
```

J24 · lun 14 déc · 6 h · cible 39,7 %

Objectif : interface CRM. **Tâches** : liste des leads avec filtres et recherche (2,5 h) · kanban des deals par étape (2,5 h) · export CSV des données autorisées (1 h). **Livrable** : CRM utilisable.

- [ ] La recherche et l'export ne renvoient que les données de l'entreprise active et du rôle
- [ ] L'export est journalisé dans l'audit

```
JOUR 24 — UI CRM. Sources : spec §5.3, §11 (audit des exports), §13
critère 1 (export inclus).
Liste des leads (filtres projet, étape, source, responsable ; recherche
nom/email/téléphone ; pagination serveur), kanban des deals avec
changement d'étape par glisser-déposer passant par le service, export
CSV respectant RLS + audit. Tests Playwright : recherche et export
n'exposent rien d'une autre entreprise.
```

J25 · mar 15 déc · 6 h · cible 41,3 %

Objectif : adaptateur CRM. **Tâches** : interface `CrmAdapter` (connect, validate, send, receive, refresh, disconnect) (1,5 h) · adaptateur interne + export/import CSV (2 h) · `crm_field_mappings` et `source_of_truth` par champ (1,5 h) · affichage du statut de synchro (1 h). **Livrable** : couche adaptateur extensible.

- [ ] Aucune synchronisation bidirectionnelle sans règles de conflit documentées
- [ ] Un échec de synchro est visible et rejouable sans doublon

```
JOUR 25 — ADAPTATEUR CRM. Sources : spec §6 (interface adapters), §9.1
étapes 5–6 et encadré « Fusion Leap CRM », §5.9 ; docs/integrations/crm.md.
packages/adapters/crm : interface commune, implémentation internal
(source de vérité MVP) et csv. Table crm_field_mappings (champ,
source_of_truth, direction). Job crm_sync idempotent via external_id,
statut et dernière erreur visibles, bouton « réessayer ». Si le CRM
Fusion Leap a une API documentée (J2), crée seulement le squelette de
son adaptateur, sans synchro bidirectionnelle.
```

J26 · mer 16 déc · 6 h · cible 42,9 %

Objectif : tests d'intégration du domaine ventes. **Tâches** : scénario T2 automatisé (2,5 h) · dédup inter-entreprises (1 h) · tests de rôles sur le CRM (1,5 h) · couverture et corrections (1 h). **Livrable** : suite verte.

- [ ] T2 : 100 leads, assignation et rapports cohérents
- [ ] Critère §13-3 démontré

```
JOUR 26 — TESTS VENTES. Sources : spec §13 critères 2-4, scénario 2.
Automatise T2 : import de leads-100.csv, vérification des comptes
(créés, fusionnés, rejetés), assignation, chiffres affichés. Ajoute :
même email dans deux entreprises, Sales Agent qui tente de lire un lead
non assigné, Analyst qui tente une transition d'étape. Rapport de
couverture ; comble les trous sur packages/domain (> 80 %).
```

J27 · jeu 17 déc · 6 h · cible 44,4 % · Checkpoint C2 / Gate G2

Objectif : prouver le parcours lead → deal. **Tâches** : test E2E formulaire → lead → assignation → deal gagné (2,5 h) · corrections (2 h) · audit de conformité Phase 2 (1,5 h). **Livrable** : `docs/checkpoints/C2.md`.

- [ ] Critère §13-4 démontré avec historique d'activités
- [ ] Aucun écart bloquant à l'audit

```
JOUR 27 — CHECKPOINT C2 / GATE G2. Sources : spec §12 (condition :
parcours lead → deal fonctionnel), §13 critères 3-4.
Test Playwright de bout en bout : soumission du formulaire public,
création et dédup, assignation, activités, passage en deal, deal
gagné, visible dans la chronologie. Simule un échec de synchro CRM et
son rejeu. Exécute le prompt d'audit de conformité (§15 du plan) sur
les Phases 1-2. Rédige docs/checkpoints/C2.md avec GO/NO-GO.
```

8. Phase 3 — Canaux (J28–J41)

La Phase 3 branche l'e-mail et WhatsApp derrière une chaîne de contrôles commune avant envoi, et les réunit dans une boîte de réception unique. Gate G3 : envoi, réception et arrêt sûrs, y compris webhook dupliqué, panne temporaire du fournisseur et désabonnement en cours d'envoi.

J28 · ven 18 déc · 6 h · cible 46,0 %

Objectif : intégrations, canaux, secrets. **Tâches** : `integrations` et `channels` + RLS (2 h) · stockage chiffré des secrets via `secret_ref` (2 h) · écran « Intégrations » sans affichage de secret brut (2 h). **Livrable** : socle d'intégration.

- [ ] Aucun jeton n'est lisible via la Data API ni par un Company Admin (spec §2)
- [ ] Rotation d'un secret testée

```
JOUR 28 — INTÉGRATIONS ET SECRETS. Sources : spec §2 (Company Admin ne
voit pas les secrets bruts), §6 (adapters), §7 (integrations, channels),
§11 ; ADR 004.
Migrations integrations (provider, status, external_account_id, scopes,
secret_ref, last_sync_at) et channels (type, integration_id,
sender_identity, status, project_scope). Module secrets côté serveur
uniquement (Vault ou chiffrement AES-GCM avec clé hors base) : put,
get, rotate, delete par secret_ref. UI Intégrations : statut, scopes,
dernier sync, déconnexion ; jamais le secret. Tests : lecture via
Data API impossible, rotation.
```

J29 · lun 21 déc · 6 h · cible 47,6 %

Objectif : réception générique des webhooks. **Tâches** : `webhook_events` + unicité `(provider, external_event_id)` (1,5 h) · route `/api/webhooks/[provider]` : taille max, signature, résolution du tenant par compte fournisseur (2,5 h) · ACK rapide + job de traitement (1 h) · tests (1 h). **Livrable** : pipeline webhook réutilisable.

- [ ] Un webhook à signature invalide est rejeté et journalisé
- [ ] Un webhook rejoué n'est traité qu'une fois

```
JOUR 29 — PIPELINE WEBHOOK. Sources : spec §8 (webhooks sans session,
signature, tenant via identité du compte fournisseur, enregistrement
puis traitement différé ; figure 7), §11.
Migration webhook_events. Route générique POST /api/webhooks/[provider]
: limite de taille, vérification de signature déléguée à l'adaptateur
(timestamp si disponible), résolution de l'organisation par
external_account_id dans integrations (JAMAIS par un champ libre du
payload), insert idempotent, réponse 200 rapide, job
process_webhook. Tests : signature invalide, rejeu, payload trop gros,
compte inconnu.
```

J30 · mar 22 déc · 6 h · cible 49,2 %

Objectif : conversations et messages. **Tâches** : `conversations` et `messages` + RLS (2 h) · trigger de cohérence `organization_id` message = conversation (1 h) · unicité `provider_message_id` par organisation (1 h) · services d'état et d'assignation (2 h). **Livrable** : modèle de messagerie.

- [ ] Un message ne peut pas porter une `organization_id` différente de sa conversation
- [ ] Un Sales Agent ne lit que les conversations qui lui sont assignées

```
JOUR 30 — CONVERSATIONS ET MESSAGES. Sources : spec §7 (conversations,
messages), §7.1 (messages héritent de l'organisation, cohérence vérifiée
en base), §9.3 (external_message_id unique).
Migrations conversations (contact_id, channel_id, assignee_id, state,
last_message_at, handled_by human|ai_suggest) et messages (direction,
provider_message_id, body, status, sent_at, received_at, error_code,
campaign_id, segment_snapshot_id, idempotency_key). Trigger qui
impose messages.organization_id = conversations.organization_id.
Unique (organization_id, channel_id, provider_message_id). RLS par rôle.
```

J31 · mer 23 déc · 6 h · cible 50,8 %

Objectif : envoi e-mail derrière la chaîne de contrôles. **Tâches** : `packages/domain/outbound` : chaîne des 6 contrôles de la figure 9 (2 h) · adaptateur e-mail (send) du fournisseur ADR 008 (2,5 h) · handler worker `send_message` (1,5 h). **Livrable** : premier e-mail de test envoyé.

- [ ] Le contrôle final est exécuté juste avant l'appel fournisseur, dans le worker
- [ ] Statut `accepted` distinct de `delivered`

```
JOUR 31 — ENVOI EMAIL. Sources : spec §9.2, figure 9 (consentement →
do_not_contact → template → limites → contrôle final → appel), §5.7
(approbation humaine) ; ADR 008, docs/integrations/email.md.
Implémente assertCanSend(message) : approbation présente, consentement
pour la finalité, do_not_contact, suppression, template si requis,
quota et rate limit. Adaptateur email (interface §6) pour le
fournisseur retenu, mocké par MSW en test. Handler send_message :
relit l'état en base, assertCanSend, appelle le fournisseur, stocke
provider_message_id et status=accepted. Aucun envoi depuis une route HTTP.
```

J32 · jeu 24 déc · 6 h · cible 52,4 %

Objectif : e-mails entrants et statuts. **Tâches** : webhook de statuts (delivered, bounced, complained) (2 h) · réception entrante et rattachement par `Message-ID` / `In-Reply-To` (2,5 h) · file des cas ambigus (1,5 h). **Livrable** : fil e-mail complet.

- [ ] Une réponse e-mail se rattache à la bonne conversation
- [ ] Un bounce met à jour le message et le contact

```
JOUR 32 — EMAIL ENTRANT ET STATUTS. Sources : spec §9.2 (rattachement
par identifiants et en-têtes, plan pour cas ambigus, pas de delivered
sur simple acceptation API).
Adaptateur email : receive() et parseStatusEvent(). Handlers worker
pour statuts (delivered, bounced, complained → do_not_contact pour
complaint) et messages entrants (threading Message-ID / In-Reply-To /
References, sinon par email du contact ; si ambigu → conversation
« à trier »). Tests avec payloads réels de la doc fournisseur.
```

J33 · ven 25 déc · 6 h · cible 54,0 %

Objectif : désabonnement et suppression. **Tâches** : lien de désabonnement signé + en-tête `List-Unsubscribe` (2 h) · page publique de désabonnement (1,5 h) · mise à jour immédiate consent + `do_not_contact` + annulation des jobs en attente (2 h) · tests (0,5 h). **Livrable** : désabonnement conforme.

- [ ] Après désabonnement, le message suivant du même contact est bloqué (critère §13-5)
- [ ] Les jobs en attente pour ce contact passent en `cancelled`

```
JOUR 33 — DÉSABONNEMENT. Sources : spec §9.2 (lien de désabonnement,
identité d'expéditeur claire), §9.3 (révocation immédiate, arrêt des
jobs), §13 critère 5.
Jeton de désabonnement signé (HMAC, organisation + contact + canal),
en-têtes List-Unsubscribe et List-Unsubscribe-Post, page publique
AR/EN. Service revokeConsent() : consent_event revoked, do_not_contact,
annulation transactionnelle des jobs send_message en attente pour ce
contact. Test : désabonnement entre deux envois d'une même série.
```

J34 · lun 28 déc · 6 h · cible 55,6 %

Objectif : connexion WhatsApp Cloud API. **Tâches** : adaptateur WhatsApp `connect` / `validate` (compte, numéro, permissions) (2,5 h) · vérification du webhook (challenge) et signature `X-Hub-Signature-256` (2 h) · écran de connexion (1,5 h). **Livrable** : numéro de test connecté.

- [ ] Le webhook de test Meta est vérifié et signé correctement
- [ ] Le tenant est résolu par l'identifiant du compte WhatsApp Business

```
JOUR 34 — CONNEXION WHATSAPP. Sources : spec §9.3 (Cloud API officielle,
vérification de propriété, état du numéro, permissions, webhook et
signature), §15 ; docs/integrations/whatsapp.md.
Relis la doc Meta officielle actuelle (lien §15) et cite la version de
l'API. packages/adapters/whatsapp : connect (stockage du jeton via
secret_ref), validate (numéro, statut, permissions), verifyWebhook
(hub.challenge) et verifySignature (X-Hub-Signature-256 avec l'app
secret). Écran de connexion du canal. Tests avec payloads de la doc.
```

J35 · mar 29 déc · 6 h · cible 57,1 %

Objectif : réception WhatsApp. **Tâches** : parsing messages entrants (texte, médias en référence) (2,5 h) · statuts sent/delivered/read/failed (1,5 h) · dédup par `external_message_id` (1 h) · tests (1 h). **Livrable** : messages entrants dans les conversations.

- [ ] Un webhook dupliqué n'affiche le message qu'une fois (critère §13-6)
- [ ] Les statuts de lecture s'affichent

```
JOUR 35 — RÉCEPTION WHATSAPP. Sources : spec §9.3 (external_message_id
unique, statuts de livraison et de lecture), §13 critère 6.
Handlers worker : messages entrants → contact (création si nouveau, via
normalizePhone) → conversation → message ; statuts → mise à jour du
message. Médias : stocker uniquement la référence, téléchargement
différé vers Storage privé. Tests : même webhook deux fois, statuts
hors ordre (read avant delivered).
```

J36 · mer 30 déc · 6 h · cible 58,7 %

Objectif : envoi WhatsApp conforme. **Tâches** : `message_templates` (E5) et synchronisation (2 h) · fenêtre de conversation : texte libre dedans, template approuvé dehors (2 h) · envoi via la chaîne de contrôles (1,5 h) · tests (0,5 h). **Livrable** : envoi WhatsApp sûr.

- [ ] Un message initié hors fenêtre sans template approuvé est bloqué
- [ ] `template_id`, version et langue sont enregistrés sur le message

```
JOUR 36 — ENVOI WHATSAPP. Sources : spec §9.3 (template approuvé hors
fenêtre, consentement, template_id/version/language, échec stocké),
figure 9 ; ADR 007 (E5).
Migration message_templates (provider, name, version, language,
category, status). Synchronisation des templates via l'adaptateur.
Étends assertCanSend : calcule la fenêtre depuis le dernier message
entrant, exige un template approuvé sinon. Adaptateur send() texte et
template. Stocke les codes d'erreur Meta. Tests des deux cas.
```

J37 · jeu 31 déc · 6 h · cible 60,3 %

Objectif : boîte de réception unifiée. **Tâches** : liste des conversations avec filtres canal/état/responsable (2 h) · vue fil avec statuts (2 h) · assignation et changement d'état (1,5 h) · temps réel via abonnement filtré par organisation (0,5 h). **Livrable** : inbox fonctionnelle.

- [ ] E-mail et WhatsApp apparaissent dans la même liste
- [ ] Le temps réel ne diffuse rien d'une autre entreprise

```
JOUR 37 — INBOX UNIFIÉE. Sources : spec §5.5, §9.3 (statuts dans la
boîte unifiée).
Pages Inbox : liste (filtres canal, état, responsable, non lus), fil
(messages, statuts, consentement du contact visible), assignation et
états (open, pending, closed). Temps réel Supabase Realtime avec RLS
active ou rafraîchissement par polling si la RLS Realtime n'est pas
garantie : documente le choix dans un ADR. Tests Playwright.
```

J38 · lun 4 jan · 6 h · cible 61,9 %

Objectif : réponses et verrou de prise en main. **Tâches** : `POST /api/conversations/:id/replies` → job + message `queued` (2 h) · verrou humain/IA (2 h) · composeur avec état d'envoi (2 h). **Livrable** : réponse depuis l'inbox.

- [ ] La réponse part via le worker, jamais dans la requête HTTP
- [ ] Un agent et une suggestion IA ne peuvent pas répondre simultanément

```
JOUR 38 — RÉPONSES. Sources : spec §8 (POST /api/conversations/:id/
replies → job + message queued), §9.3 (prise en main par un employé,
pas de réponse IA et humaine en même temps).
Endpoint de réponse (withService, Idempotency-Key) qui crée le message
queued et le job send_message. Verrou de conversation (lock_owner,
lock_expires_at) : un humain qui prend la main désactive les
suggestions IA. Composeur : statut queued → accepted → delivered/read
ou failed avec raison. Tests de concurrence.
```

J39 · mar 5 jan · 6 h · cible 63,5 %

Objectif : exploitation des envois. **Tâches** : limites de débit par fournisseur et organisation (2 h) · tableau dead letter avec rejeu (1,5 h) · arrêt d'urgence par organisation et par canal (1,5 h) · alertes d'échec d'intégration (1 h). **Livrable** : contrôles d'exploitation.

- [ ] L'arrêt d'urgence bloque tout envoi en moins d'un cycle de worker
- [ ] Un job en dead letter est visible et rejouable

```
JOUR 39 — EXPLOITATION DES ENVOIS. Sources : spec §6 (limites par
fournisseur et entreprise, dead_letter), §11 (plan d'arrêt d'urgence
par entreprise et canal), §5.8 (alertes), §13 critère 10.
Rate limiter (token bucket en base) par (provider, organisation).
Page Exploitation : jobs échoués, dead letter, rejeu manuel audité.
Interrupteur send_paused par organisation et par canal, vérifié dans
assertCanSend. Alertes in-app sur échec répété d'intégration.
Runbook docs/runbooks/emergency-stop.md.
```

J40 · mer 6 jan · 6 h · cible 65,1 %

Objectif : scénarios de canaux. **Tâches** : T3 (série e-mail + désabonnement en cours) (2 h) · T4 (WhatsApp client + template, webhook dupliqué, panne 5xx) (2,5 h) · corrections (1,5 h). **Livrable** : suite verte.

- [ ] Critères §13-5 et §13-6 démontrés
- [ ] Une panne 5xx du fournisseur produit des retries puis un succès, sans doublon

```
JOUR 40 — SCÉNARIOS CANAUX. Sources : spec §13 critères 5, 6, 10 et
scénarios 3 et 4.
Automatise T3 et T4 avec fournisseurs simulés (MSW) : série email de
5 contacts consentants, désabonnement du 3e en cours ; conversation
WhatsApp initiée par le client puis par template ; même webhook reçu
deux fois ; fournisseur qui renvoie 503 deux fois puis 200. Vérifie
les compteurs, les statuts et l'absence de doublon.
```

J41 · jeu 7 jan · 6 h · cible 66,7 % · Checkpoint C3 / Gate G3

Objectif : valider les canaux de bout en bout sur les comptes de test réels. **Tâches** : test manuel e-mail et WhatsApp sur comptes sandbox (2 h) · corrections (2 h) · audit de conformité Phase 3 (2 h). **Livrable** : `docs/checkpoints/C3.md`.

- [ ] Envoi, réception et arrêt sûrs démontrés sur comptes de test officiels
- [ ] Aucun secret ni corps de message dans les logs (vérifié par recherche)

```
JOUR 41 — CHECKPOINT C3 / GATE G3. Sources : spec §12 (condition :
envoi, réception et arrêt sûrs), §11.
Guide-moi pas à pas pour un test manuel sur les comptes sandbox réels
(email, WhatsApp) : envoi approuvé, réponse entrante, statuts, arrêt
d'urgence. Puis recherche automatisée de fuites dans les logs (jetons,
corps de messages). Exécute le prompt d'audit de conformité (§15 du
plan). Rédige docs/checkpoints/C3.md avec GO/NO-GO.
```

9. Phase 4 — Croissance (J42–J53)

La Phase 4 livre les campagnes avec approbation, les agents IA en mode brouillon et le tableau de bord sourcé. Gate G4 : une campagne complète est auditable de la création au résultat, et aucun brouillon IA ne part sans validation humaine.

J42 · ven 8 jan · 6 h · cible 68,3 %

Objectif : modèle de campagne et machine d'états. **Tâches** : `campaigns` + `campaign_assets` versionnés + RLS (2,5 h) · machine d'états pure dans `packages/domain` (2,5 h) · tests exhaustifs des transitions (1 h). **Livrable** : domaine campagne.

- [ ] Toute transition non listée au §10 est refusée
- [ ] `paused`, `rejected`, `failed` gérés

```
JOUR 42 — CAMPAGNES : MODÈLE. Sources : spec §7 (campaigns,
campaign_assets), §10 (états draft → pending_approval → approved →
scheduled → running → completed ; paused, rejected, failed ; figure 12).
Migrations campaigns (project_id, objective, channel, status,
segment_snapshot_id, scheduled_at, budget, currency) et campaign_assets
(kind, content, version, approval_status, approved_by, approved_at).
Machine d'états pure (table de transitions + gardes par rôle).
Tests table-driven sur toutes les paires (état, action, rôle).
```

J43 · mar 12 jan · 6 h · cible 69,8 %

Objectif : segments et snapshots. **Tâches** : `audience_segments` avec conditions en DSL validé (2 h) · estimation de taille (1 h) · `segment_snapshots` figés + hash (2 h) · UI de segment (1 h). **Livrable** : audiences ciblables.

- [ ] Un segment exclut toujours `do_not_contact` et les contacts sans consentement pour le canal
- [ ] Le snapshot ne change plus après création

```
JOUR 43 — SEGMENTS. Sources : spec §7 (audience_segments, snapshot avant
envoi), §10 (snapshot + hash d'envoi) ; ADR 007 (E9).
Migration audience_segments (définition jsonb validée par zod :
projet, étape, source, pays, langue, tags) et segment_snapshots
(contact_ids figés, count, hash, created_at, immuable par trigger).
Compilation sécurisée DSL → SQL paramétré filtré par organization_id.
Exclusions obligatoires : do_not_contact, consentement absent pour le
canal. Estimation de taille. UI de construction.
```

J44 · mer 13 jan · 6 h · cible 71,4 %

Objectif : soumission et approbation. **Tâches** : `submit` et `approve` (avec snapshot) (2 h) · retour automatique en `pending_approval` si contenu ou segment modifié (1,5 h) · écran de revue avec diff de version (2 h) · audit (0,5 h). **Livrable** : flux d'approbation.

- [ ] Modifier un asset approuvé remet la campagne en `pending_approval` (critère §13-7)
- [ ] Seul un rôle habilité approuve ; l'approbation est auditée

```
JOUR 44 — APPROBATION. Sources : spec §8 (POST /api/campaigns,
/:id/submit, /:id/approve), §10 (toute modification après approbation →
pending_approval), §2 (Marketing Manager), §13 critère 7.
Endpoints create, submit, approve via withService. approve fige le
snapshot du segment. Trigger/service : toute modification d'asset ou
de segment d'une campagne approved crée une nouvelle version et
repasse en pending_approval. Écran de revue avec diff entre versions.
Audit de chaque transition. Tests rôle par rôle.
```

J45 · ven 15 jan · 6 h · cible 73,0 %

Objectif : planification et exécution. **Tâches** : `schedule` avec fuseau IANA (1,5 h) · job « fan-out » qui crée un job d'envoi par destinataire avec `idempotency_key` (2 h) · pause, reprise, arrêt (1,5 h) · compteurs planifié/envoyé/échoué (1 h). **Livrable** : campagne exécutable.

- [ ] Arrêter une campagne annule les jobs non envoyés
- [ ] Les compteurs correspondent exactement aux messages en base

```
JOUR 45 — EXÉCUTION DE CAMPAGNE. Sources : spec §8 (/:id/schedule, heure
IANA, canal connecté, jobs après contrôle des consentements), §10
(campaign_id, segment_snapshot_id, idempotency_key par message ; arrêt
si consentement retiré ou intégration désactivée), §13 critère 7.
schedule → job campaign_fanout à scheduled_at ; fan-out par lots,
idempotency_key = hash(campaign, version, contact). Chaque envoi passe
par assertCanSend. Pause/reprise/arrêt transactionnels. Arrêt
automatique si l'intégration est désactivée. Compteurs calculés depuis
messages. Tests : arrêt à mi-parcours, rejeu du fan-out.
```

J46 · lun 18 jan · 6 h · cible 74,6 %

Objectif : UTM et attribution. **Tâches** : générateur UTM normalisé (1,5 h) · attribution lead → campagne (1,5 h) · import manuel des dépenses marqué « importé » (1,5 h) · calcul du coût par lead avec « non disponible » (1,5 h). **Livrable** : mesure de campagne.

- [ ] Une dépense importée n'est jamais présentée comme « en direct »
- [ ] Le coût par lead affiche « non disponible » sans dépense fiable

```
JOUR 46 — UTM ET ATTRIBUTION. Sources : spec §4.1 (plan Google/Meta,
dépense saisie manuellement en MVP, ne pas présenter comme en direct),
§10 (UTM unifiés, coût = dépense / leads sur période et canal connus,
« non disponible » sinon).
Générateur d'URL UTM par campagne. Attribution des leads entrants via
utm_campaign. Table campaign_spend (période, canal, montant, devise,
source=manual_import). Fonction costPerLead() qui renvoie
{ value, source } ou { unavailable }. Tests sur les cas limites.
```

J47 · mar 19 jan · 6 h · cible 76,2 %

Objectif : socle IA gouverné. **Tâches** : adaptateur fournisseur IA (1,5 h) · `agent_runs` + registre de prompts versionnés (2 h) · budget d'usage et délai par organisation (1,5 h) · minimisation des données dans le contexte (1 h). **Livrable** : runtime d'agents.

- [ ] Chaque run enregistre `prompt_version`, modèle, tokens, coût estimé, sources, statut
- [ ] Aucun secret ni donnée d'une autre entreprise dans le contexte (test)

```
JOUR 47 — RUNTIME IA. Sources : spec §9.5 (contexte isolé par entreprise
et projet, pas de secrets, prompt_version, modèle, coût, sources,
décision du relecteur, budget et timeout par entreprise ; données de
test uniquement ; figure 11), §7 (agent_runs).
Adaptateur AI (interface commune, fournisseur configurable). Migration
agent_runs. Registre prompts/ versionné dans le dépôt. runAgent() :
construit un contexte minimal via des fonctions de lecture filtrées
par organization_id, applique budget et timeout, enregistre le run.
Aucun outil d'écriture. Bascule « données réelles interdites » par
environnement. Tests d'isolation du contexte.
```

J48 · mer 20 jan · 6 h · cible 77,8 %

Objectif : agents Campaign et Research. **Tâches** : Campaign Agent (copies e-mail/WhatsApp/annonce) (2,5 h) · Research Agent (synthèse audience, idées de test) (2 h) · intégration dans l'éditeur de campagne (1,5 h). **Livrable** : brouillons IA dans les campagnes.

- [ ] Un brouillon IA entre en `draft`, jamais `approved`
- [ ] Les sources du contexte sont affichées à côté du brouillon

```
JOUR 48 — CAMPAIGN & RESEARCH AGENTS. Sources : spec §9.5 (tableau des
agents : entrées autorisées, sorties, limite d'action), §5.7.
Campaign Agent : entrées objectif, offre, identité de marque, canal,
contraintes → variantes de contenu enregistrées comme campaign_assets
en draft. Research Agent : description produit, segment, sources
ajoutées par l'utilisateur → synthèse interne. POST /api/agents/draft
(brouillon seulement, avec sources). UI : bouton « Générer un
brouillon », sources affichées, édition humaine obligatoire.
```

J49 · jeu 21 jan · 6 h · cible 79,4 %

Objectif : agents Qualification et Reply. **Tâches** : Lead Qualification Agent (score, raisons, questions manquantes) (2 h) · Reply Agent (suggestion dans l'inbox) (2 h) · traitement des messages entrants comme données non fiables (2 h). **Livrable** : IA dans le CRM et l'inbox.

- [ ] La qualification ne rejette jamais un lead automatiquement
- [ ] La suggestion de réponse exige un clic humain pour partir

```
JOUR 49 — QUALIFICATION & REPLY AGENTS. Sources : spec §9.5 (Lead
Qualification : suggestion seulement ; Reply : envoi humain seulement ;
messages clients et pages externes = entrées non fiables), §9.3 (verrou
humain/IA) ; ADR 007 (E4 knowledge_items).
Migration knowledge_items. Qualification : score + raisons + questions
manquantes affichés sur la fiche lead, jamais de changement d'étape
automatique. Reply : suggestion dans le composeur, respecte le verrou
du J38. Délimite le contenu non fiable dans le prompt et ignore toute
instruction qu'il contient. Tests avec messages piégés.
```

J50 · ven 22 jan · 6 h · cible 81,0 %

Objectif : passerelle MCP (optionnelle) et tests d'injection. **Tâches** : ADR 009 MCP maintenant ou plus tard (1 h) · si oui, outils lecture seule `get_lead_context`, `draft_reply` (2,5 h) · suite d'injection de prompt (2,5 h). **Livrable** : décision MCP + suite anti-injection.

- [ ] Un prompt caché dans un message entrant ne déclenche aucune action ni fuite (critère §13-8)
- [ ] Aucun outil MCP n'a accès à `service_role` ni aux secrets fournisseurs

```
JOUR 50 — MCP ET ANTI-INJECTION. Sources : spec §1 (MCP n'est pas une
file ni un moteur de permissions), §6 (outils nommés, lecture seule
d'abord, écriture via permissions + approbation + audit, pas de service
role), §13 critère 8 ; spec §15 (lien MCP).
1. ADR 009 : MCP nécessaire dans le MVP ? Recommande et attends mon OK.
2. Si OK : passerelle interne avec get_lead_context et draft_reply en
   lecture seule, authentifiée par contexte tenant.
3. Suite de tests d'injection : instructions cachées en arabe, français
   et anglais dans des emails/WhatsApp entrants (demande de données
   d'une autre entreprise, d'envoi, de changement d'étape).
```

J51 · lun 25 jan · 6 h · cible 82,5 %

Objectif : métriques et tableau de bord. **Tâches** : événements de mesure normalisés (1,5 h) · job d'agrégation `metric_daily` idempotent (2 h) · tableau de bord par entreprise et projet avec source et date de mise à jour (2,5 h). **Livrable** : tableau de bord.

- [ ] Chaque chiffre indique son type (direct, importé, calculé) et sa dernière mise à jour
- [ ] Un webhook dupliqué ne double pas une conversion

```
JOUR 51 — MÉTRIQUES ET TABLEAU DE BORD. Sources : spec §1.1 objectif 5,
§7 (metric_daily), §10 (événements lead_created, qualified,
meeting_booked, deal_won/lost, message_sent/delivered/replied,
app_installed ; source, date, type ; pas de double comptage).
Table metric_events (dédupliquée par clé naturelle) et job
aggregate_metrics quotidien idempotent vers metric_daily. Tableau de
bord : leads, qualification, conversations, réunions, deals, coût par
lead, avec badge de source et horodatage. Tests de non-double-comptage.
```

J52 · mar 26 jan · 6 h · cible 84,1 %

Objectif : Analyst Agent et flux d'activité. **Tâches** : Analyst Agent sur métriques agrégées (2 h) · flux d'activité de l'organisation (2 h) · centre d'alertes (intégrations, dead letter, quotas) (2 h). **Livrable** : couche d'analyse.

- [ ] L'Analyst Agent ne lit que des agrégats et ne modifie aucun budget
- [ ] Les alertes sont visibles des rôles concernés uniquement

```
JOUR 52 — ANALYST AGENT ET ALERTES. Sources : spec §9.5 (Analyst :
métriques agrégées et leur source, ne modifie pas les budgets), §5.8.
Analyst Agent : entrée = metric_daily + sources, sortie = explications
et recommandations, citées. Flux d'activité filtré par rôle. Centre
d'alertes : échecs d'intégration, dead letter, quota à 80/100 %,
jeton expirant. Tests de visibilité par rôle.
```

J53 · mer 27 jan · 6 h · cible 85,7 % · Checkpoint C4 / Gate G4

Objectif : prouver une campagne complète et auditable. **Tâches** : test E2E de campagne (2,5 h) · corrections (2 h) · audit de conformité Phase 4 (1,5 h). **Livrable** : `docs/checkpoints/C4.md`.

- [ ] Critères §13-7 et §13-8 démontrés
- [ ] La piste d'audit reconstitue qui a créé, modifié, approuvé, planifié et arrêté

```
JOUR 53 — CHECKPOINT C4 / GATE G4. Sources : spec §12 (condition :
campagne complète auditable), §13 critères 7–8.
Test E2E : création → brouillon IA → édition → soumission → approbation
→ modification (retour en revue) → réapprobation → planification →
envoi partiel → pause → arrêt ; compteurs exacts ; reconstitution
complète depuis audit_logs et agent_runs. Exécute l'audit de
conformité (§15 du plan). Rédige docs/checkpoints/C4.md avec GO/NO-GO.
```

10. Phase 5 — Pilote SaaS (J54–J62)

La Phase 5 rend la plateforme utilisable par d'autres entreprises sans intervention en base, la durcit et la fait passer en recette sur 2 à 3 tenants pilotes. Gate G5 : les 10 critères d'acceptation du §13 sont démontrés.

J54 · jeu 28 jan · 6 h · cible 87,3 %

Objectif : onboarding en libre-service. **Tâches** : assistant d'inscription (organisation, slug/sous-domaine, fuseau, langue) (2,5 h) · identité de marque, politique de consentement, heures ouvrées (2 h) · checklist de démarrage (1,5 h). **Livrable** : onboarding complet.

- [ ] Une nouvelle entreprise crée un projet, invite un membre et connecte un canal sans toucher à la base (critère §13-9)
- [ ] Aucun modèle ni client de Fusion Leap n'est visible d'une autre entreprise

```
JOUR 54 — ONBOARDING SAAS. Sources : spec §4.4 (inscription, espace,
sous-domaine, invitations, projets, canaux, marque, consentement,
heures ouvrées ; rien de Fusion Leap visible par défaut), §13 critère 9.
Assistant en étapes : organisation + slug unique (validation, mots
réservés), fuseau et langue, marque (logo dans Storage tenant,
couleurs), texte et version de consentement par défaut, heures ouvrées.
Checklist « premiers pas » : projet, invitation, canal. Test E2E d'une
inscription complète par un utilisateur neuf.
```

J55 · ven 29 jan · 6 h · cible 88,9 %

Objectif : connecteur Salla selon l'hypothèse du §14. **Tâches** : mode marketing (projets `salla_app` / `salla_theme` sans permission boutique) vérifié (1 h) · squelette OAuth Salla en sandbox : install, refresh, uninstall (3 h) · webhooks `app.installed` / `app.uninstalled` (2 h). **Livrable** : adaptateur Salla isolé, désactivé par défaut.

- [ ] Aucune donnée de boutique n'est lue sans OAuth et scopes explicites
- [ ] À la désinstallation, la synchro s'arrête et la politique de rétention s'applique

```
JOUR 55 — CONNECTEUR SALLA. Sources : spec §4.2, §4.3, §9.4 (séparation
marketing / données marchand, store_id, refresh sans supposer la
validité, app.installed / app.uninstalled, pas de chiffres estimés
présentés comme confirmés ; figure 10), §14 Q4 ; docs/integrations/salla.md.
Relis la doc Salla officielle (liens §15). Implémente l'adaptateur
derrière un feature flag désactivé : OAuth sandbox, stockage des jetons
par secret_ref, refresh proactif, table salla_stores (store_id ↔
organisation/projet), webhooks via le pipeline du J29. Si l'hypothèse
§14 est maintenue, s'arrêter là : pas de lecture de commandes.
```

J56 · lun 1 fév · 6 h · cible 90,5 %

Objectif : console Platform Admin. **Tâches** : liste des organisations, plans, statuts, consommation (2 h) · sessions de support limitées dans le temps, motif et audit (E7) (3 h) · tests (1 h). **Livrable** : console admin.

- [ ] Un Platform Admin ne voit aucune donnée client hors session de support active
- [ ] Chaque session est auditée et expire automatiquement

```
JOUR 56 — CONSOLE PLATFORM ADMIN. Sources : spec §2 (Platform Admin :
accès aux données client uniquement via session de support enregistrée,
limitée dans le temps, après autorisation), §10, §11 ; ADR 007 (E7).
Migration support_sessions (organization_id, admin_id, reason,
approved_by, starts_at, expires_at). Politiques RLS : platform_admin
n'accède aux tables tenant que si une session active existe. Console :
organisations, plan, statut, consommation. Audit de chaque ouverture
et de chaque accès. Tests d'expiration.
```

J57 · mar 2 fév · 6 h · cible 92,1 %

Objectif : durcissement sécurité. **Tâches** : en-têtes de sécurité et CSP (1 h) · CSRF sur les mutations par cookie (1 h) · contrôle du type et de la taille des uploads (1 h) · checklist de production Supabase + revue GRANT (2 h) · scan des dépendances (1 h). **Livrable** : `docs/security/hardening-report.md`.

- [ ] Chaque point de la checklist de production Supabase est coché ou justifié
- [ ] Aucune vulnérabilité haute ou critique ouverte

```
JOUR 57 — DURCISSEMENT. Sources : spec §11 (limites de débit, validation,
CSRF, en-têtes de sécurité, uploads), §15 (checklist de production
Supabase) ; docs/security/threat-model.md.
Applique les en-têtes (CSP, HSTS, X-Content-Type-Options,
Referrer-Policy, frame-ancestors), CSRF pour les mutations à base de
cookie, vérification magic bytes + taille des fichiers. Parcours la
checklist officielle Supabase point par point. Revue de tous les GRANT
et de la Data API. pnpm audit. Revérifie chaque menace du threat model
et rédige hardening-report.md.
```

J58 · mer 3 fév · 6 h · cible 93,7 %

Objectif : observabilité, sauvegarde, runbooks. **Tâches** : suivi des erreurs et alertes (2 h) · exercice de sauvegarde et restauration réel (2 h) · runbooks : incident, arrêt d'urgence, rotation de secrets, restauration (2 h). **Livrable** : dossier `docs/runbooks/` complet.

- [ ] Une restauration a été exécutée et chronométrée (critère §13-10)
- [ ] Une alerte arrive en cas de dead letter ou d'échec d'intégration

```
JOUR 58 — OBSERVABILITÉ ET RESTAURATION. Sources : spec §11 (sauvegarde
et restauration testées, surveillance, alertes, arrêt d'urgence), §13
critère 10.
Branche le suivi des erreurs (web + worker) avec request_id, alertes
sur dead letter, file bloquée, intégration en échec. Guide-moi pour un
exercice de restauration sur staging et chronomètre-le. Rédige
docs/runbooks/ : incident, emergency-stop, secret-rotation, restore,
worker-stuck, provider-outage.
```

J59 · jeu 4 fév · 6 h · cible 95,2 %

Objectif : staging et tenants pilotes. **Tâches** : déploiement staging (web + worker + migrations) (2 h) · 2 à 3 entreprises pilotes avec données de test (2 h) · scénarios T1 et T5 sur staging (2 h). **Livrable** : staging opérationnel.

- [ ] Les migrations s'appliquent sur une base vierge et sur staging sans erreur
- [ ] T1 et T5 passent sur staging

```
JOUR 59 — STAGING ET PILOTES. Sources : spec §6 (environnements
séparés), §12 Phase 5 (2–3 entreprises pilotes séparées), §13
scénarios 1 et 5. Utilise la skill engineering:deploy-checklist si
disponible.
Prépare la checklist de déploiement, déploie web et worker sur staging,
applique les migrations, crée 3 organisations pilotes (dont Fusion Leap)
avec données fictives. Rejoue T1 et T5 contre staging. Rapport des
écarts entre local et staging.
```

J60 · ven 5 fév · 6 h · cible 96,8 %

Objectif : suite de recette complète. **Tâches** : suite E2E couvrant les 10 critères et les 5 scénarios (4 h) · rapport de recette (2 h). **Livrable** : `docs/acceptance/report.md`.

- [ ] Chaque critère §13 a un test automatisé ou une preuve manuelle documentée
- [ ] Tous les tests passent sur staging

```
JOUR 60 — RECETTE. Sources : spec §13 intégral.
Construis la suite tests/acceptance/ : un fichier par critère 1 à 10 et
par scénario T1 à T5, exécutable contre staging. Pour ce qui ne peut
pas être automatisé (comptes Meta réels, restauration), fournis une
procédure manuelle avec preuves attendues. Produis
docs/acceptance/report.md : critère, test, résultat, preuve, statut.
```

J61 · lun 8 fév · 6 h · cible 98,4 %

Objectif : corrections et audit final. **Tâches** : correction des anomalies de recette (4 h) · audit de conformité global (2 h). **Livrable** : audit final.

- [ ] Aucune anomalie bloquante ou majeure ouverte
- [ ] Les 15 invariants du §1.1 sont vérifiés dans le code

```
JOUR 61 — CORRECTIONS ET AUDIT FINAL. Sources : docs/acceptance/report.md,
spec intégrale, §1.1 du plan (invariants).
Corrige les anomalies par gravité (bloquant, majeur, mineur) sans
élargir le périmètre. Puis exécute le prompt d'audit de conformité
(§15 du plan) sur l'ensemble du dépôt, invariant par invariant, avec
chemins de fichiers en preuve. Liste la dette restante.
```

J62 · mar 9 fév · 6 h · cible 100 % · Gate G5

Objectif : clôture du MVP. **Tâches** : recette finale avec vous (2 h) · notes de version et CHANGELOG (1 h) · documentation de passation (2 h) · rétrospective et backlog post-MVP (1 h). **Livrable** : version `v1.0.0-mvp` taguée.

- [ ] Les 10 critères §13 sont « Réussi » dans le rapport de recette
- [ ] Documentation complète : architecture, API, runbooks, ADR, guide utilisateur

```
JOUR 62 — CLÔTURE MVP / GATE G5. Sources : spec §12 (condition : critères
d'acceptation réussis), §13.
Prépare la démonstration de recette (script pas à pas des 10
critères). Rédige RELEASE_NOTES.md, finalise CHANGELOG.md, docs/handover.md
(architecture, exploitation, limites connues, décisions §14 encore
ouvertes). Propose le backlog post-MVP classé (facturation,
connecteurs publicitaires, Salla données marchand, synchro CRM
bidirectionnelle). Tague v1.0.0-mvp après mon accord.
```

11. Stratégie de tests

Cinq niveaux de tests, tous exécutés en CI à chaque PR ; les tests RLS ont le même poids que les tests unitaires car l'isolation est le critère n°1 de la spec.

Niveau	Ce qu'il prouve	Outil	Où	Seuil / règle
Unitaires	Règles métier pures : normalisation, déduplication, machine d'états, permissions, `assertCanSend`, coût par lead	Vitest	`packages/domain`, `packages/permissions`	Couverture ≥ 80 % sur ces packages ; tests table-driven
RLS / base	Aucune fuite inter-tenant, rôles respectés, triggers d'intégrité, immutabilité de l'audit	pgTAP via `supabase test db`, ou Vitest avec JWT par utilisateur	`supabase/tests`	Chaque table tenant : matrice utilisateur × entreprise × CRUD ; obligatoire dès sa migration
Intégration	Services + base + worker ensemble ; adaptateurs avec fournisseurs simulés	Vitest + Supabase local + MSW	`apps/*/tests/integration`	Chaque endpoint du §8 et chaque handler de job
Contrat fournisseurs	Parsing des webhooks et réponses réels	Fixtures copiées de la doc officielle	`packages/adapters/*/fixtures`	Une fixture par type d'événement utilisé
E2E / acceptation	Parcours utilisateur et critères §13	Playwright (AR RTL et EN)	`tests/e2e`, `tests/acceptance`	Les 10 critères + scénarios T1–T5 avant G5

Règles transverses

- **Test d'abord pour la sécurité** : tout contrôle d'accès s'écrit avec son test négatif (ce qui doit être refusé) avant le code.
- **Deux entreprises dans chaque fixture** : le seed de test contient toujours au moins deux organisations et un utilisateur commun, pour que chaque test puisse vérifier l'absence de fuite.
- **Fournisseurs simulés en CI**, comptes sandbox réels uniquement au checkpoint C3 et à la recette finale (J59–J62).
- **Aucune donnée client réelle** dans les tests ni dans les prompts IA (spec §9.5).

- **Un bug corrigé = un test de non-régression** ajouté dans le même commit.

Prompt de rédaction de tests (réutilisable)

```
Écris d'abord les tests pour [fonctionnalité], avant tout code de
production. Couvre : cas nominal, entrées invalides (zod), rôle non
autorisé, membre d'une autre entreprise (doit échouer), rejeu idempotent,
concurrence si pertinent. Cite pour chaque test la section de spec qu'il
prouve. Montre-moi la liste des tests, attends mon OK, puis implémente
jusqu'à ce qu'ils passent, sans modifier un test pour le faire passer.
```

12. Suivi quotidien et avancement réel

L'avancement réel ne compte que le travail **validé** : une tâche n'entre dans le pourcentage que lorsque tous ses critères sont cochés et ses tests verts. Code écrit mais non testé = 0 %.

```
\text{Avancement réel} = \frac{\sum \text{heures estimées des tâches
validées}}{378\ \text{h}} \times 100
```

Écart (réel − cible du jour)	Statut	Action
⩾ −2 points	Vert	Continuer
−2 à −5 points	Orange	Utiliser le tampon de fin de phase ; signaler au point hebdomadaire
< −5 points ou gate échouée	Rouge	Procédure de blocage (§13) : réduire, découper ou décaler, avec accord du product owner

Modèle de journal (`docs/progress/journal.md`)

```
## J07 — jeu 19 nov 2026 — Phase 1
Cible : 12,7 % | Réel : 12,2 % | Écart : −0,5 pt | Statut : Vert
### Tâches
| Tâche | Estimé | Passé | Validée |
| Politiques RLS noyau | 2,5 h | 3 h | oui |
| Revue GRANT / Data API | 1 h | 0,5 h | oui |
| Harnais test:rls | 2,5 h | 2,5 h | non (2 tests rouges) |
### Tests : 84 unit ✓ · 31 RLS (29 ✓ 2 ×) · 0 E2E
### Écarts à la spec : aucun
### Décisions / ADR : —
### Blocages : JWT de test expirés en CI → piste : durée longue en local
### Demain : terminer harnais (1 h) puis J8
```

Prompt de démarrage (chaque matin, nouvelle session)

```
DÉMARRAGE DE SESSION — Jour [N].
1. Relis CLAUDE.md, docs/plan/development-plan.md (Jour [N]) et les 3
   dernières entrées de docs/progress/journal.md.
2. Lance pnpm lint && pnpm typecheck && pnpm test && pnpm test:rls et
   donne-moi l'état (vert/rouge, nombre de tests).
3. Résume en 5 lignes : où en est le projet, ce qui reste du jour
   précédent, les risques ouverts.
N'écris aucun code. Attends le prompt du jour.
```

Prompt de clôture (chaque soir)

```
CLÔTURE — Jour [N].
1. Liste les tâches du jour [N] avec : estimé, temps réel approximatif,
   validée oui/non (validée = critères cochés ET tests verts).
2. Lance la suite complète et rapporte les chiffres exacts.
3. Calcule l'avancement réel = somme des heures estimées des tâches
   validées depuis J0 / 378 h. Compare à la cible du jour [N] du plan.
4. Ajoute l'entrée du jour dans docs/progress/journal.md au format du
   modèle, mets à jour CHANGELOG.md.
5. Commit « chore(progress): journal J[N] » et ouvre/mets à jour la PR.
Ne déclare jamais validée une tâche dont un test est rouge.
```

Prompt de rapport hebdomadaire (vendredi)

```
RAPPORT HEBDOMADAIRE — semaine du [date].
À partir de docs/progress/journal.md, rédige en français, pour la
direction, docs/progress/weekly/[AAAA-SS].md :
- Avancement réel vs cible (points), statut vert/orange/rouge
- Livré cette semaine (fonctionnalités visibles, pas de jargon)
- Critères §13 désormais démontrés
- Risques et blocages, décisions attendues de la direction (§14)
- Objectifs de la semaine prochaine
Une page maximum, chiffres exacts uniquement.
```

13. Dépendances, risques et blocages

Le chemin critique passe par les comptes de test externes : sans numéro WhatsApp officiel au 28 décembre (J34), la Phase 3 glisse. Les demandes d'accès partent donc dès la Phase 0.

13.1 Dépendances externes

Dépendance	Nécessaire le	À lancer au plus tard	Responsable	Plan B si absente
Réponses Q1–Q5 du §14	J1 (10 nov)	J0	Product owner	Hypothèse exécutive du §14, notée en ADR
Projet Supabase staging + prod	J5 (16 nov)	J3	Responsable technique	Supabase local jusqu'à J59
Domaine e-mail avec SPF/DKIM/DMARC	J31 (23 déc)	J2	Propriétaire du domaine	Domaine de test du fournisseur, envoi limité
Compte fournisseur e-mail (ADR 008)	J31 (23 déc)	J3	Responsable technique	Sandbox du fournisseur ou serveur SMTP local (Mailpit)

Dépendance	Nécessaire le	À lancer au plus tard	Responsable	Plan B si absente
App Meta + numéro WhatsApp Cloud API de test	J34 (28 déc)	J2	Product owner	Tests sur fixtures + MSW ; validation réelle reportée au J59
Templates WhatsApp approuvés par Meta	J36 (30 déc)	J28	Marketing	Tester uniquement dans la fenêtre de conversation
Clé API du fournisseur IA + conditions de conservation	J47 (19 jan)	J40	Product owner	Fournisseur simulé ; IA désactivée en pilote
Compte partenaire Salla (sandbox)	J55 (29 jan)	J40	Product owner	Connecteur laissé derrière le feature flag
Documentation du CRM Fusion Leap	J25 (15 déc)	J2	Product owner	CRM interne seul + CSV (prévu par la spec §9.1)

13.2 Risques techniques

Risque	Probabilité	Impact	Prévention	Réaction
Fuite inter-tenant par une politique RLS oubliée	Moyenne	Critique	Test RLS obligatoire à chaque migration ; script CI qui échoue si une table publique n'a pas RLS	Arrêt du développement de fonctionnalités jusqu'à correction
Récursion RLS sur `organization_members`	Haute	Élevé	Fonctions `security definer` dans le schéma `private` (J6)	Revue avec l'exemple de la doc Supabase

Risque	Probabilité	Impact	Prévention	Réaction
Délais d'approbation Meta	Haute	Élevé	Demande au J2	Plan B fixtures ; démonstration réelle au J59
Envoi en double (retry, webhook rejoué)	Moyenne	Élevé	`idempotency_key`, unicités, contrôle final dans le worker	Dead letter + correction + test de non-régression
Injection de prompt via message entrant	Moyenne	Élevé	Aucun outil d'écriture pour l'IA ; contenu non fiable délimité ; suite de tests J50	Désactiver l'agent concerné par organisation
Évolution des API fournisseurs ou de Next.js/Supabase	Moyenne	Moyen	Versions épinglées ; relecture de la doc officielle avant chaque connecteur (spec §15)	ADR de mise à jour, migration isolée
L'agent de codage s'écarte de la spec ou invente	Haute	Moyen	`CLAUDE.md`, plan avant code, audit de conformité à chaque gate	Revert du commit, prompt de recadrage (§16)
Sous-estimation des phases 3 et 4	Moyenne	Moyen	Tampon en fin de phase	Découper : reporter MCP (J50) et Analyst Agent (J52) après le MVP

13.3 Procédure en cas de blocage

1. **Moins de 2 h perdues** : prompt de débogage (§16), puis on continue.
2. **Une demi-journée perdue** : noter dans le journal, consommer le tampon de fin de phase.
3. **Plus d'un jour ou gate échouée** : prompt de replanification ci-dessous, décision du product owner, mise à jour du plan et des cibles.

```
REPLANIFICATION — nous sommes au Jour [N], avancement réel [X] %, cible
[Y] %. Blocage : [description].
Analyse : 1) cause racine ; 2) impact sur les jours suivants et sur les
gates ; 3) trois options chiffrées (heures) : réduire le périmètre
(sans toucher aux invariants §1.1 ni aux critères §13), découper la
tâche, décaler le planning. Recommande une option. Ne modifie pas le
plan avant ma décision ; ensuite mets à jour development-plan.md et
note la décision dans un ADR.
```

14. Documentation continue

La documentation est écrite le jour même où le code change, jamais en fin de projet : c'est une étape du protocole de tâche de `CLAUDE.md`, vérifiée à chaque clôture.

Document	Emplacement	Créé	Mis à jour
Spécification (référence figée)	`docs/spec/`	J0	Jamais modifiée ; les écarts passent par ADR
Analyse et écarts	`docs/analysis/`	J0	Quand un écart est tranché
Plan de développement	`docs/plan/development-plan.md`	J0	Après chaque replanification validée
ADR (Contexte, Décision, Alternatives, Conséquences, Section de spec)	`docs/adr/NNN-*.md`	J1	À chaque décision ; statut Proposed → Accepted
Architecture, ERD, flux de données	`docs/architecture/`	J1–J2	À chaque migration qui change le modèle
Fiches d'intégration	`docs/integrations/`	J2	Avant chaque connecteur
Référence API (endpoints, schémas zod, erreurs)	`docs/api/`	J11	À chaque endpoint ajouté ou modifié
Sécurité : menaces, durcissement	`docs/security/`	J3	J57 et à chaque nouvelle surface

Document	Emplacement	Créé	Mis à jour
Runbooks d'exploitation	`docs/runbooks/`	J39	J58, puis à chaque incident
Journal, rapports hebdomadaires	`docs/progress/`	J0	Chaque soir, chaque vendredi
Rapports de checkpoint	`docs/checkpoints/C1…C4.md`	J15	À chaque gate
Recette	`docs/acceptance/report.md`	J60	J61–J62
CHANGELOG, notes de version, passation	racine, `docs/handover.md`	J4	Chaque jour, J62

Prompt de mise à jour documentaire

```
DOCUMENTATION — compare le code actuel à docs/. Pour chaque écart
(endpoint non documenté, table absente de l'ERD, ADR contredit par le
code, runbook obsolète), liste : fichier de doc, fichier de code,
correction proposée. Applique les corrections de documentation
seulement ; si c'est le code qui contredit un ADR ou la spec, ne le
modifie pas et signale-le-moi.
```

15. Checkpoints de conformité à la spécification

Quatre checkpoints de phase (C1–C4), une recette finale et une revue hebdomadaire légère garantissent que le développement ne dérive pas de la spec. Chaque checkpoint produit un GO/NO-GO écrit ; un NO-GO bloque la phase suivante.

Checkpoint	Date	Vérifie	Condition de GO
G0 / G0′	9 et 12 nov	Compréhension, plan, ADR	« PLAN VALIDÉ » + ADR 001–008 acceptés
C1 / G1	1er déc	Isolation, rôles, socle	Critères §13-1 et §13-2 démontrés
C2 / G2	17 déc	Capture, CRM	Critères §13-3 et §13-4
C3 / G3	7 jan	Canaux, consentement, arrêt	Critères §13-5, §13-6, partie de §13-10

Checkpoint	Date	Vérifie	Condition de GO
C4 / G4	27 jan	Campagnes, IA, mesure	Critères §13-7 et §13-8
Recette / G5	9 fév	Tout le §13	10 critères « Réussi »
Revue hebdo	Chaque vendredi	Invariants §1.1 sur le code de la semaine	Aucun invariant violé

Prompt d'audit de conformité (à chaque checkpoint)

```
AUDIT DE CONFORMITÉ — Phase [N]. Tu es auditeur, pas développeur :
n'écris AUCUN code dans cette tâche.

1. Invariants : pour chacun des 15 invariants de docs/plan (§1.1),
   donne CONFORME / NON CONFORME / NON APPLICABLE, avec la preuve
   (chemin de fichier + ligne, ou test qui le démontre).
2. Architecture : compare l'arborescence et les dépendances réelles au
   §6 de la spec. Signale toute bibliothèque, service ou couche ajoutés
   sans ADR (ex. ORM, file externe, n8n, appel direct au fournisseur
   depuis une route HTTP).
3. Données : pour chaque table créée, vérifie organization_id NOT NULL,
   index, RLS activée, politiques par opération, test RLS existant.
   Requête SQL à l'appui sur la base locale.
4. Périmètre : liste tout ce qui a été construit hors MVP (§1.2 de la
   spec) et tout ce qui était prévu pour cette phase et manque.
5. Critères §13 visés par cette phase : test associé + résultat.
6. Sécurité : recherche de secrets dans le code, les logs de test et le
   bundle client ; service_role hors serveur/worker.
Rends docs/checkpoints/C[N].md avec un tableau des écarts classés
(bloquant / majeur / mineur) et une recommandation GO ou NO-GO.
```

Prompt de revue hebdomadaire (vendredi, 30 min)

```
REVUE HEBDO — examine uniquement les commits de cette semaine
(git log --since="7 days ago"). Pour chaque commit : invariant §1.1
concerné, respecté ou non, test présent ou non. Signale toute table,
endpoint ou dépendance absent de la spec et des ADR. Pas de code.
```

16. Bibliothèque de prompts récurrents

Ces prompts couvrent les situations qui reviennent hors du planning : bug, revue, recadrage, reprise de contexte. Ils complètent les prompts quotidiens sans les remplacer.

Débogage structuré

```
DÉBOGAGE — Symptôme : [erreur / comportement]. Attendu : [x].
1. Reproduis avec un test qui échoue (montre-le-moi).
2. Isole la cause en lisant le code concerné ; énonce 2–3 hypothèses
   et comment tu les vérifies.
3. Propose le correctif minimal, attends mon OK.
4. Corrige, garde le test comme non-régression, relance toute la suite.
Interdit : désactiver un test, affaiblir une politique RLS, ajouter un
try/catch silencieux.
```

Revue de code avant merge

```
REVUE DE CODE — diff de la branche [day-XX-...] vs main.
Vérifie dans l'ordre : 1) isolation tenant (organization_id, RLS,
contexte serveur) ; 2) autorisation (can() + RLS) ; 3) idempotence et
concurrence ; 4) validation des entrées ; 5) secrets et logs ;
6) tests présents et pertinents ; 7) conformité à la spec et aux ADR ;
8) lisibilité. Classe chaque remarque bloquant / à corriger / suggestion,
avec fichier:ligne. Ne modifie rien.
```

Recadrage (l'agent s'écarte de la spec)

```
STOP. Tu t'écartes de la spécification : [constat]. Relis CLAUDE.md et
la section [§x] de la spec. Explique en 3 lignes l'écart, puis propose
comment revenir à la spec (revert, correction). Ne code rien avant mon
OK. Si tu penses que la spec a tort, rédige un ADR « Proposed » au lieu
de l'ignorer.
```

Reprise après interruption ou contexte perdu

```
REPRISE — Relis CLAUDE.md, le plan (Jour [N]), les 5 dernières entrées
du journal, git log -20 et git status. Dis-moi : ce qui est terminé et
validé, ce qui est en cours et non commité, ce qui est cassé. Propose
la suite exacte. Pas de code avant mon OK.
```

Refactoring sans changement de comportement

```
REFACTORING — Cible : [module]. Objectif : [lisibilité / duplication /
performance]. Contraintes : aucun changement de comportement, aucun
test modifié, API publique inchangée. Montre le plan, attends mon OK,
procède par petits commits avec la suite verte à chaque étape.
```

Nouvelle migration de base

```
MIGRATION — Besoin : [x], section de spec : [§x]. Crée une NOUVELLE
migration (ne modifie jamais une migration appliquée) avec :
organization_id NOT NULL + index si table tenant, FK composites, RLS
activée + politiques par opération, triggers updated_at, et la
procédure de retour arrière. Ajoute les tests RLS dans le même commit.
Mets à jour docs/architecture/erd.md.
```

Lecture de la documentation officielle d'un fournisseur

```
VÉRIFICATION DOC — Avant de coder [connecteur], lis la documentation
officielle [lien de la spec §15]. Rends : version de l'API, endpoints,
format des webhooks, signature, limites, conditions de consentement,
changements récents. Cite les URL consultées. Marque « À CONFIRMER » ce
que la doc ne dit pas clairement. Mets à jour docs/integrations/[x].md.
```